

■ Trilha Completa de JavaScript para Frontend

Guia de estudos + espaços para anotações

Como usar este material

1. Estude o conteúdo original do Markdown.
2. Use as caixas verdes para fixar os conceitos.
3. Use as áreas amarelas para escrever dúvidas, exemplos e resumos.
4. Assista ao material recomendado quando precisar de uma explicação em vídeo.
5. Tente os exercícios antes de procurar soluções prontas.

Biblioteca rápida de apoio

Curso em Vídeo — curso introdutório de JavaScript e ECMAScript para iniciantes. O material oficial disponibiliza PDFs, códigos, exercícios e desafios. [Abrir material](#).

Playlist oficial no YouTube: [Curso de JavaScript e ECMAScript](#).

Para Fetch/APIs: CFB Cursos — [Aprendendo sobre FETCH para consumo de APIs em Javascript](#).

Para armazenamento: CFB Cursos — [WebStorage, localStorage e SessionStorage em Javascript](#).

Complemento para DOM/async: [JavaScript — DOM, Async/Await, CallStack e Event Loop](#).

■ Trilha Completa de JavaScript para Frontend

Apostila prática — do zero em JavaScript até consumo de APIs e projetos reais.

Pré-requisitos assumidos: lógica de programação, HTML/CSS básico-intermediário, primeiro contato com variáveis/condicionais/loops/funções/arrays em **Python**.

■■ Mapa completo da trilha

Módulo	Tema central	O que você vai aprender
1	Fundamentos do JavaScript	<code>let/const</code> , tipos de dados, operadores, template literals
2	Controle de fluxo e funções	<code>if/else</code> , loops, funções (declaração e arrow)
3	Arrays e Objetos	arrays, objetos, destructuring, spread
4	Métodos funcionais	métodos de string, <code>map</code> , <code>filter</code> , <code>find</code> , <code>forEach</code> , <code>includes</code>
5	Introdução ao DOM	<code>querySelector</code> , <code>getElementById</code> , <code>textContent</code> , <code>innerHTML</code> , <code>value</code> , <code>createElement</code> , <code>classList</code>
6	Eventos	<code>addEventListener</code> , <code>click</code> , <code>input</code> , <code>change</code> , <code>submit</code> , <code>event</code> , <code>preventDefault</code>
7	Formulários e DOM dinâmico	formulários, manipulação de classes CSS, criação dinâmica de elementos
8	Assincronismo	funções assíncronas, <code>Promise</code> , <code>async/await</code> , <code>try/catch</code>
9	Fetch, JSON e APIs	<code>fetch</code> , <code>Response</code> , <code>response.json()</code> , consumo de APIs, API Keys
10	Armazenamento no navegador	<code>localStorage</code>
11	Organização de código	<code>import/export</code> , módulos
Projetos	Consolidação	5 projetos práticos, do simples ao completo

Como usar esta apostila:

1. Siga os módulos **na ordem** — cada um assume o anterior.
2. Leia o conceito, veja o exemplo, **tente os exercícios sozinho antes de procurar a solução**.
3. A partir do Módulo 5, todo exercício deve ser feito em arquivos reais (`index.html`, `style.css`, `script.js`), nunca só no console do navegador.
4. Termine cada módulo com o **desafio de revisão**.

Legenda de dificuldade:

- Simples — aplicação direta do conceito
 - Médio — combina 2+ conceitos
 - Avançado — simula uma funcionalidade real de frontend
-

■ Estrutura de projeto (a partir do Módulo 5)

```
projeto/  
  index.html  
  style.css  
  script.js
```

Conecte os arquivos assim:

```
<!DOCTYPE html>  
<html lang="pt-br">  
  <head>  
    <meta charset="UTF-8">  
    <title>Meu projeto</title>  
    <link rel="stylesheet" href="style.css">  
  </head>  
  <body>  
    <!-- conteúdo aqui -->  
  
    <script src="script.js"></script>  
  </body>  
</html>
```

■ Coloque o `<script>` no final do `<body>` (ou use o atributo `defer`). Isso garante que o HTML já exista na tela quando o JavaScript tentar manipulá-lo. Um erro muito comum de quem está começando é colocar o `<script>` no `<head>` sem `defer` e receber `null` ao tentar pegar um elemento — porque ele ainda não existe.

Módulo 1 — Fundamentos do JavaScript

Conceito: `let` e `const`

O que é?

São as duas formas modernas de declarar variáveis em JavaScript (existe também `var`, mas é antiga e tem comportamentos confusos — evite usá-la).

Para que serve?

`let` declara uma variável que **pode ser reatribuída**. `const` declara uma variável **constante**, que não pode ser reatribuída depois de criada.

Diferença em relação ao Python: em Python você simplesmente escreve `idade = 20`. Em JavaScript você **precisa** declarar a variável com uma palavra-chave (`let` ou `const`) antes de usá-la.

Sintaxe:

```
let idade = 20;
const nome = "Rafael";

idade = 21;      // ok, let pode mudar
nome = "Ana";    // ERRO: não pode reatribuir const
```

Exemplo:

```
let pontuacao = 0;
const nomeDoJogo = "Quiz";

pontuacao = pontuacao + 10;
console.log(pontuacao, nomeDoJogo); // 10 "Quiz"
```

■ **Armadilha comum:** `const` não torna um array ou objeto "imutável" — só impede que a variável aponte para outro valor. Você ainda pode alterar o *conteúdo* de um array/objeto declarado com `const`. Veremos isso no Módulo 3.

Regra prática: comece sempre com `const`. Só troque para `let` se você já sabe que vai reatribuir aquela variável depois.

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

let é um tipo de declaração em que o dado ele pode ser atribuído um outro valor. Já o
const é um tipo de declaração em que o dado é imutável.

EX:

`const nome = "lucas"; // nome é imutável`

`let idade = 17; // idade é mutável`

Exercícios

■ Simples

Declare uma constante `nomeUsuario` com seu nome e uma variável `idade` (usando `let`) com um número. Exiba os dois no console usando `console.log`.

■ Médio

Crie um contador de visitas: declare `let visitas = 0`; e depois escreva o código que soma 1 a essa variável três vezes seguidas, mostrando o valor no console a cada soma.

■ Avançado

Declare uma variável com `const` e tente reatribuí-la em seguida. Rode o código, leia a mensagem de erro no console do navegador e explique (em um comentário) o que ela significa. Depois corrija, trocando para `let` onde fizer sentido.

Conceito: Tipos de dados

O que é?

JavaScript possui tipos primitivos: **String** (texto), **Number** (números — não existe distinção entre inteiro e decimal), **Boolean** (**true/false**), **undefined** (variável declarada mas sem valor) e **null** (ausência de valor intencional), além de **Object** e **Array** (Módulo 3).

Para que serve?

Saber o tipo de um dado evita erros comuns, como somar um número com um texto sem perceber.

Sintaxe:

```
let texto = "Olá";      // string
let numero = 10;        // number
let ativo = true;       // boolean
let semValor;           // undefined
let vazio = null;       // null

typeof texto; // "string"
```

Exemplo:

```
let idade = "20"; // string, não number!
console.log(idade + 1); // "201" -> concatenou, não somou
console.log(typeof idade); // "string"
```

■ **Armadilha comum:** o operador **+** entre **string** e **number** **concatena** em vez de somar. Diferente do Python, que geraria erro. Em JS, **"5" + 1** vira **"51"**, mas **"5" - 1** vira **4** (subtração força conversão para número).

■ PARA FIXAR

Não memorize somente nomes; pratique `typeof` e compare valores reais.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

os tipos de dados primitivos do javascript são: string (texto), number (float e int não tem distinção), null (nulo), undefined (indefinido) e boolean (True e False). Quando se usa a operação de soma **+** entre uma string e um número ocorre a concatenação, mas se houver uma operação de subtração **-** ela é executada.

EX:

```
console.log(1 + "7"); // 17 string
console.log("7" - 1) // 6 number
```

Exercícios

■ Simples

Crie variáveis de cada tipo primitivo (**string**, **number**, **boolean**, **undefined**, **null**) e use `typeof` para imprimir o tipo de cada uma no console.

■ Médio

Dado `let precoTexto = "150";` e `let quantidade = 3;`, escreva o código que calcula o valor total corretamente (convertendo `precoTexto` para número). Pesquise sobre `Number()` ou `parseInt()`.

■ Avançado

Crie três variáveis simulando dados vindos de um formulário HTML (que sempre chegam como `string`): `"25"` (idade), `"true"` (aceitou termos) e `"9.90"` (preço). Converta cada uma para o tipo correto (`number`, `boolean`, `number`) e comprove com `typeof`.

Conceito: Operadores

O que é?

Símbolos para realizar operações: aritméticos (+ - / % *), de comparação (== === != !== > < >= <=) e lógicos (&& || !).

Para que serve?

São a base de qualquer lógica condicional ou cálculo.

Sintaxe:

```
5 + 3;      // 8
5 === "5";  // false (compara tipo e valor)
5 == "5";   // true (só compara valor, ignora tipo)
true && false; // false
true || false; // true
```

Exemplo:

```
let a = 10;
let b = "10";
console.log(a == b); // true
console.log(a === b); // false
```

■■ **Armadilha comum — a mais importante do módulo:** existem `==` (igualdade solta, converte tipos antes de comparar) e `===` (igualdade estrita, compara tipo e valor). **Sempre prefira `===` e `!==`.** Usar `==` é fonte clássica de bugs difíceis de encontrar.

■ PARA FIXAR

Antes de executar, tente prever o resultado da expressão.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Escreva expressões usando cada operador aritmético (+ - / % *) com dois números e mostre os resultados no console.

■ Médio

Compare `10 == "10"`, `10 === "10"`, `0 == false` e `null == undefined` no console. Anote em comentários o resultado de cada uma e tente explicar o porquê.

■ Avançado

Crie uma verificação de "usuário pode entrar na balada": a pessoa precisa ter `idade >= 18` e `temIngresso === true`. Monte essa condição com variáveis e operadores lógicos, imprimindo `true` ou `false` conforme os valores mudam.

Conceito: Template literals

O que é?

Forma moderna de criar strings usando crase (``` ```) em vez de aspas, permitindo interpolar variáveis diretamente no texto.

Para que serve?

Evita concatenação manual com `+`, deixando o código mais legível — muito usado mais adiante para montar HTML dinamicamente.

Sintaxe:

```
const nome = "Rafael";
const mensagem = `Olá, ${nome}!`;
```

Exemplo:

```
const produto = "Notebook";
const preco = 3200;
console.log(`O produto ${produto} custa R$ ${preco}.`);
```

Compare com a forma antiga: `"O produto " + produto + " custa R$ " + preco + "."`

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie variáveis `nome` e `cidade` e monte uma frase com template literals: "Rafael mora em São Paulo".

■ Médio

Dadas as variáveis `produto`, `preco` e `quantidade`, monte uma string com template literals mostrando: "3x Notebook = R\$ 9600" (o total deve ser calculado dentro do `${}`).

■ Avançado

Crie uma "nota fiscal" em texto usando template literals com quebras de linha (dentro da crase dá pra apertar Enter), mostrando nome do cliente, 2-3 produtos com preços e o total geral.

■ Desafio de revisão — Módulo 1

Crie um script que:

1. Use `const` para dados que não mudam (nome de uma loja) e `let` para dados que mudam (total de uma compra).
2. Simule 3 produtos com preços vindos como `string` (ex: "29.90") e converta para `number`.
3. Calcule o total da compra.
4. Use um operador de comparação para verificar se o total passou de R\$ 100 (frete grátis).
5. Exiba um resumo final usando template literals.

Módulo 2 — Controle de fluxo e funções

Pré-requisito: Módulo 1.

■ COMO ESTUDAR ESTE MÓDULO

Foco: Não avance enquanto variáveis, tipos e operadores ainda parecerem mecânicos. Faça pequenos testes no console e observe os valores.

Prática: Dica: tente prever o resultado antes de executar o código.

Material: YouTube PT-BR: Curso em Vídeo — base de JavaScript, especialmente variáveis, tipos e operadores.

Conceito: `if` / `else`

O que é?

Estrutura condicional — igual à lógica que você já conhece de Python, mudando apenas a sintaxe.

Para que serve?

Executar blocos de código diferentes dependendo de uma condição.

Diferenças em relação ao Python:

- Precisa de `{ }` para delimitar blocos (Python usa indentação).
- A condição fica entre parênteses `( )`.
- Não existe `elif`, usa-se `else if`.
- Não termina linha com `:`.

Sintaxe:

```
if (condicao) {  
  // executa se verdadeiro  
} else if (outraCondicao) {  
  // executa se a primeira for falsa e essa for verdadeira  
} else {  
  // executa se nenhuma condição anterior for verdadeira  
}
```

Exemplo:

```
const idade = 16;  
  
if (idade >= 18) {  
  console.log("Maior de idade");  
} else {  
  console.log("Menor de idade");  
}
```

Existe também o operador ternário, uma forma compacta de `if/else` para uma única expressão:

```
const status = idade >= 18 ? "Maior" : "Menor";
```

■ **Armadilha comum:** valores como `0`, `""` (string vazia), `null`, `undefined` e `NaN` são considerados "falsy" (se comportam como `false` dentro de um `if`). Todo o resto é "truthy". Isso costuma confundir quem vem de linguagens mais rígidas.

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie uma variável `nota` e escreva um `if/else` que imprima `"Aprovado"` se a nota for maior ou igual a 6, e `"Reprovado"` caso contrário.

■ Médio

Crie uma classificação de faixa etária com `if/else if/else`: `"criança"` (até 12), `"adolescente"` (13 a 17), `"adulto"` (18 ou mais).

■ Avançado

Simule uma validação de senha simples: a senha deve ter pelo menos 8 caracteres (dica: `.length`) e conter pelo menos um número (pode simular isso verificando se `senha.includes` algum dígito — não se preocupe em fazer isso perfeito, é uma introdução). Imprima mensagens diferentes para cada motivo de falha.

Conceito: Loops

O que é?

Estruturas de repetição. As principais em JavaScript são `for`, `while` e o moderno `for...of` (para percorrer arrays).

Para que serve?

Repetir um bloco de código várias vezes, ou percorrer os elementos de uma coleção.

Sintaxe:

```
// for clássico
for (let i = 0; i < 5; i++) {
  console.log(i);
}

// while
let contador = 0;
while (contador < 5) {
  console.log(contador);
  contador++;
}

// for...of - percorre valores de um array (muito usado no dia a dia)
const frutas = ["maçã", "banana", "uva"];
for (const fruta of frutas) {
  console.log(fruta);
}
```

Exemplo:

```
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) {
    console.log(`${i} é par`);
  }
}
```

■ **Armadilha comum:** confundir **for...of** (percorre **valores**) com **for...in** (percorre **índices/chaves** — geralmente usado com objetos, e raramente com arrays). Para arrays no dia a dia, prefira **for...of** ou os métodos do Módulo 4 (**map**, **filter**, **forEach**).

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Use um **for** clássico para imprimir os números de 1 a 20 no console.

■ Médio

Dado o array `const numeros = [4, 15, 8, 23, 42, 7];`, use um **for...of** para imprimir apenas os números maiores que 10.

■ Avançado

Simule uma "tabela de preços com desconto progressivo": para quantidades de 1 a 10 unidades de um produto de R\$ 50, use um loop para calcular e imprimir o preço total de cada quantidade, aplicando 5% de desconto extra a cada unidade acima de 5 (ex: 6 unidades = 5% off, 7 = 10% off, etc — use sua criatividade na regra, o importante é praticar loop + condicional juntos).

Conceito: Funções

O que é?

Blocos de código reutilizáveis. Em JavaScript existem duas formas principais: **function declaration** e **arrow function** (mais moderna e muito comum em código atual).

Para que serve?

Evitar repetição de código e organizar a lógica em blocos nomeados e reutilizáveis.

Sintaxe:

```
// function declaration (parecida com Python, mas sem "def" e com chaves)
function somar(a, b) {
  return a + b;
}

// arrow function (moderna, muito usada)
const somar2 = (a, b) => {
  return a + b;
};

// arrow function com retorno implícito (uma linha só)
const somar3 = (a, b) => a + b;
```

Exemplo:

```
function saudacao(nome) {
  return `Olá, ${nome}!`;
}

console.log(saudacao("Ana")); // "Olá, Ana!"
```

■ ■ **Armadilha comum:** esquecer o **return**. Sem ele, a função sempre retorna **undefined**, mesmo que ela "pareça" estar calculando algo internamente.

Quando usar cada forma: para funções nomeadas e independentes, **function** funciona bem e é fácil de ler. Arrow functions são muito usadas como argumento de outras funções (você verá isso constantemente no Módulo 4 com **map**, **filter** etc.) e em eventos (Módulo 6).

■ PARA FIXAR

Pense em função como uma pequena tarefa: recebe algo, processa e pode devolver algo.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie uma função `calcularArea(base, altura)` que retorna a área de um retângulo. Teste chamando-a com valores diferentes.

■ Médio

Crie uma função `classificarNota(nota)` que retorna "Aprovado", "Recuperação" ou "Reprovado" (defina as faixas). Reescreva a mesma função como arrow function.

■ Avançado

Crie uma função `calcularFrete(peso, distancia)` que aplica regras diferentes: se peso > 10kg, cobra R\$ 2 por kg extra; se distância > 100km, adiciona uma taxa fixa. A função deve combinar `if/else` e retornar o valor final calculado. Teste com pelo menos 4 combinações diferentes de peso e distância.

■ Desafio de revisão — Módulo 2

Crie uma função `gerarBoletim(notas)` que recebe um array de notas (ex: `[7, 5, 9, 4]`), usa um loop para percorrer o array, e para cada nota usa um `if/else` para classificar. Ao final, imprima no console quantas notas foram aprovadas e quantas foram reprovadas, usando template literals no resultado.

Módulo 3 — Arrays e Objetos

Pré-requisito: Módulos 1 e 2.

■ COMO ESTUDAR ESTE MÓDULO

Foco: Aqui começa a ponte entre JavaScript e a página. Pense no DOM como a representação da estrutura HTML que o JavaScript consegue acessar.

Prática: Dica: pratique seleção de elementos e eventos em páginas pequenas antes de juntar tudo em projetos.

Material: YouTube PT-BR: Curso em Vídeo — aulas de DOM e Eventos DOM.

[Abrir material recomendado](#)

Conceito: Arrays

O que é?

Lista ordenada de valores, igual às listas do Python — a sintaxe é bem parecida.

Para que serve?

Armazenar coleções de dados: lista de produtos, tarefas, usuários, etc.

Sintaxe:

```
const frutas = ["maçã", "banana", "uva"];

frutas[0];           // "maçã"
frutas.length;       // 3
frutas.push("pera"); // adiciona no final
frutas.pop();         // remove o último
```

Exemplo:

```
const tarefas = ["Estudar JS", "Fazer exercícios"];
tarefas.push("Revisar DOM");
console.log(tarefas); // ["Estudar JS", "Fazer exercícios", "Revisar DOM"]
```

■ **Armadilha comum:** como vimos no Módulo 1, `const tarefas = [...]` não impede de usar `.push()`, `.pop()` ou alterar `tarefas[0]`. O que `const` impede é fazer `tarefas = outroArray`. Isso confunde muita gente.

■ PARA FIXAR

Pratique sempre com exemplos pequenos antes de usar arrays complexos.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um array `cores` com 5 cores. Imprima a primeira, a última (usando `.length`) e o tamanho do array.

■ Médio

Crie um array `carrinho` vazio. Adicione 3 produtos com `.push()`. Remova o último com `.pop()`. Imprima o carrinho após cada operação.

■ Avançado

Crie um array de 6 números. Sem usar os métodos do Módulo 4 ainda (só loop + **if**), calcule e imprima: a soma de todos, a média, e quantos são maiores que a média.

Conceito: Objetos

O que é?

Estrutura de dados com pares **chave: valor** — equivalente aos dicionários (**dict**) do Python, mas com sintaxe de `{ chave: valor }` e acesso via `objeto.chave` ou `objeto["chave"]`.

Para que serve?

Representar entidades do mundo real com várias propriedades: um produto, um usuário, um personagem de uma API.

Sintaxe:

```
const produto = {
  nome: "Notebook",
  preco: 3200,
  disponivel: true
};

produto.nome; // "Notebook"
produto["preco"]; // 3200
produto.desconto = 10; // adiciona nova propriedade
```

Exemplo:

```
const usuario = {
  nome: "Rafael",
  idade: 20,
  cursos: ["JS", "HTML", "CSS"]
};

console.log(`${usuario.nome} tem ${usuario.idade} anos.`);
```

Objetos podem conter arrays, e arrays podem conter objetos — essa combinação é **extremamente comum** ao trabalhar com APIs (Módulo 9):

```
const produtos = [
  { nome: "Mouse", preco: 50 },
  { nome: "Teclado", preco: 120 }
];

console.log(produtos[0].nome); // "Mouse"
```

■ **Armadilha comum:** usar `.` para acessar uma chave que tem espaço ou vem de uma variável. Nesses casos use colchetes: `produto["chave com espaço"]` ou `produto[variavelComNomeDaChave]`.

■ PARA FIXAR

Pense em objeto como uma entidade com propriedades e, quando necessário, comportamentos.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um objeto `pessoa` com `nome`, `idade` e `cidade`. Imprima uma frase usando os três valores com template literals.

■ Médio

Crie um array de 3 objetos `produto` (cada um com `nome` e `preco`). Use um loop `for...of` para imprimir uma linha por produto no formato `"Mouse - R$ 50"`.

■ Avançado

Crie um objeto `carrinho` com uma propriedade `itens` (array de objetos produto, cada um com `nome`, `preco` e `quantidade`). Escreva a lógica (loop) que calcula o valor total do carrinho somando `preco * quantidade` de cada item.

Conceito: Destructuring

O que é?

Sintaxe que permite "extrair" valores de arrays ou propriedades de objetos diretamente em variáveis, sem precisar acessar posição por posição.

Para que serve?

Deixa o código mais curto e legível — muito usado ao lidar com dados de APIs e com parâmetros de função.

Sintaxe:

```
// objeto
const produto = { nome: "Mouse", preco: 50 };
const { nome, preco } = produto;

// array
const cores = ["vermelho", "verde", "azul"];
const [primeira, segunda] = cores;
```

Exemplo:

```
const usuario = { nome: "Ana", idade: 22, cidade: "SP" };
const { nome, cidade } = usuario;
console.log(`${nome} - ${cidade}`); // "Ana - SP"
```

Também funciona direto nos parâmetros de uma função:

```
function mostrarProduto({ nome, preco }) {  
  console.log(`${nome}: R$ ${preco}`);  
}  
mostrarProduto({ nome: "Mouse", preco: 50 });
```

■ PARA FIXAR

Pergunte: quais valores eu quero retirar de dentro deste objeto/array?

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um objeto `livro` com `titulo`, `autor` e `ano`. Use destructuring para extrair `titulo` e `autor` em variáveis separadas e imprima-as.

■ Médio

Crie um array `coordenadas = [10, 20]`. Use destructuring para extrair em `x` e `y`. Depois crie uma função `distanciaDaOrigem` que recebe um array `[x, y]` já desestruturado no parâmetro e retorna `x + y` (simplificação didática).

■ Avançado

Dado um array de objetos `pedidos`, cada um com `{ cliente, valor, status }`, use um loop com destructuring diretamente na variável do `for...of` (ex: `for (const { cliente, valor, status } of pedidos)`) para imprimir apenas os pedidos com `status === "pendente"`.

Conceito: Spread operator

O que é?

O operador `...` (três pontos), usado para "espalhar" os elementos de um array ou as propriedades de um objeto.

Para que serve?

Copiar arrays/objetos sem alterar o original, juntar arrays, ou criar um novo objeto com algumas propriedades alteradas — fundamental para trabalhar sem "mutar" dados diretamente, prática comum em frontend.

Sintaxe:

```
const original = [1, 2, 3];
const copia = [...original, 4]; // [1, 2, 3, 4], original intacto

const produto = { nome: "Mouse", preco: 50 };
const produtoComDesconto = { ...produto, preco: 40 }; // sobrescreve só o preço
```

Exemplo:

```
const tarefasAntigas = ["Estudar", "Praticar"];
const tarefasNovas = [...tarefasAntigas, "Revisar"];
console.log(tarefasNovas); // ["Estudar", "Praticar", "Revisar"]
console.log(tarefasAntigas); // continua ["Estudar", "Praticar"]
```

■ ■ **Armadilha comum:** fazer `const copia = original`; **não cria uma cópia** — cria outra variável apontando para o **mesmo** array/objeto na memória. Alterar `copia` também altera `original`. O spread resolve isso criando uma cópia de fato (rasa/"shallow", mas suficiente para o que você vai precisar por enquanto).

■ PARA FIXAR

Compare o resultado de copiar uma estrutura com e sem spread.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um array `numeros = [1, 2, 3]`. Use spread para criar `numerosCopia` adicionando o número 4 ao final, sem alterar o array original. Imprima os dois.

■ Médio

Crie um objeto `configuracao = { tema: "claro", idioma: "pt" }`. Use spread para criar uma nova versão com `tema: "escuro"`, mantendo `idioma` igual, sem alterar o objeto original.

■ Avançado

Crie dois arrays de tarefas (`tarefasCasa` e `tarefasTrabalho`). Use spread para juntá-los em um único array `todasTarefas`, e depois use spread novamente para adicionar uma nova tarefa `"Descansar"` no início da lista (dica: `[novoItem, ...arrayExistente]`).

■ Desafio de revisão — Módulo 3

Crie um array de objetos representando um "estoque de loja" (pelo menos 4 produtos, cada um com **nome**, **preço** e **quantidade**). Depois:

1. Use um loop para calcular o valor total do estoque (**preço** * **quantidade** de cada item, somado).
2. Use destructuring dentro do loop para acessar as propriedades.
3. Use spread para criar uma cópia do array de estoque com um novo produto adicionado, sem alterar o array original.

Módulo 4 — Métodos funcionais de String e Array

Pré-requisito: Módulos 1, 2 e 3. Estes métodos vão substituir a maioria dos loops manuais que você escreveu até agora — são a forma "idiomática" de trabalhar com arrays em JavaScript moderno.

■ COMO ESTUDAR ESTE MÓDULO

Foco: Arrays e métodos como map/filter/find são fundamentais para interfaces dinâmicas. Treine com dados simples antes de usar APIs.

Prática: Dica: sempre tente explicar em português o que entra e o que sai de cada método.

Material: YouTube PT-BR: Curso em Vídeo — fundamentos e exercícios de JavaScript.

[Abrir material recomendado](#)

Conceito: Métodos de strings

O que é?

Funções prontas para manipular texto: **.length**, **.toUpperCase()**, **.toLowerCase()**, **.trim()**, **.split()**, **.slice()**, **.replace()**, **.includes()**.

Para que serve?

Formatar, validar e transformar texto — essencial em qualquer interface que lida com dados digitados pelo usuário.

Sintaxe:

```
const texto = " Olá, Mundo! ";
texto.trim();           // "Olá, Mundo!" (remove espaços das pontas)
texto.toUpperCase();    // " OLÁ, MUNDO! "
texto.includes("Mundo"); // true
texto.replace("Mundo", "JS"); // " Olá, JS! "
"a,b,c".split(",");     // ["a", "b", "c"]
```

Exemplo:

```
const email = " Rafael@Email.com ";
const emailLimp = email.trim().toLowerCase();
console.log(emailLimp); // "rafael@email.com"
```

■ **Armadilha comum:** métodos de string **não alteram a string original** (strings são imutáveis em JS) — eles sempre **retornam uma nova string**. Se você não guardar o retorno em uma variável, a alteração se perde.

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Dada `const frase = " javascript é ótimo "`, remova os espaços das pontas e deixe a primeira letra maiúscula (pode usar `.charAt(0).toUpperCase() + frase.slice(1)`).

■ Médio

Crie uma função `validarEmail(email)` que retorna `true` se o texto contém "@" e ".", usando `.includes()`.

■ Avançado

Crie uma função `gerarUsername(nomeCompleto)` que recebe um nome como "Rafael Souza Lima" e retorna um username no formato "rafael.souza.lima" (minúsculo, espaços trocados por pontos). Use `.toLowerCase()`, `.trim()` e `.split()/.replace()`.

Conceito: `forEach`

O que é?

■ **Método** de array que executa uma função para **cada elemento**, sem gerar um novo array.

Para que serve?

Substituir o `for...of` quando você só precisa "fazer algo" com cada item (ex: imprimir, ou mais pra frente, criar elementos na tela).

Sintaxe:

```
array.forEach((item, indice) => {  
  // código  
});
```

Exemplo:

```
const nomes = ["Ana", "Bruno", "Carla"];
nomes.forEach((nome, i) => {
  console.log(`${i + 1} - ${nome}`);
});
```

■ **Armadilha comum:** `forEach` não retorna nada útil (sempre `undefined`). Se você precisa de um novo array como resultado, use `map` (a seguir).

■ PARA FIXAR

`forEach` executa uma ação para cada item; ele não foi feito para montar um novo array.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Use `forEach` para imprimir cada item de um array de tarefas no console, numerado.

■ Médio

Use `forEach` para imprimir apenas os números pares de um array de números (combine com `if` dentro do `forEach`).

■ Avançado

Dado um array de objetos `produtos` (com `nome` e `preco`), use `forEach` para imprimir uma "nota fiscal" no console, linha por linha, e acumular o total em uma variável `let total = 0;` declarada fora do `forEach`.

Conceito: `map`

O que é?

Metodo de array que **transforma cada elemento** e retorna um **novo array** com o mesmo tamanho.

Para que serve?

Sempre que você precisa gerar uma nova lista a partir de outra (ex: transformar uma lista de preços em uma lista de preços com desconto) — é o método mais usado ao gerar HTML dinamicamente a partir de dados (Módulo 5 em diante).

Sintaxe:

```
const novoArray = array.map((item) => {  
  return item; // valor transformado  
});
```

Exemplo:

```
const precos = [100, 200, 300];  
const precosComDesconto = precos.map((preco) => preco * 0.9);  
console.log(precosComDesconto); // [90, 180, 270]
```

■ ■ **Armadilha comum:** esquecer o **return** dentro do **map** quando se usa chaves **{ }** na arrow function. Sem **return**, cada posição do novo array vira **undefined**.

■ PARA FIXAR

Map transforma: normalmente entra um item e sai outro item.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Dado um array de números, use **map** para criar um novo array com cada número multiplicado por 2.

■ Médio

Dado um array de objetos **produtos** (**nome**, **preco**), use **map** para criar um novo array só com os nomes (**"Mouse"**, **"Teclado"**, **...**).

■ Avançado

Dado um array de objetos **produtos** (**nome**, **preco**), use **map** para criar um novo array de objetos, onde cada um tem as mesmas propriedades, mas com um novo campo **precoComDesconto** (10% off), sem alterar o array original (use **spread!**).

Conceito: **filter**

O que é?

Método de array que retorna um **novo array** apenas com os elementos que satisfazem uma condição.

Para que serve?

Filtrar listas: produtos disponíveis, tarefas concluídas, resultados de busca — a base de qualquer funcionalidade de "filtro" em uma interface.

Sintaxe:

```
const filtrado = array.filter((item) => {  
  return condicao; // true mantém o item, false descarta  
});
```

Exemplo:

```
const numeros = [4, 15, 8, 23, 42, 7];  
const maioresQue10 = numeros.filter((n) => n > 10);  
console.log(maioresQue10); // [15, 23, 42]
```

■ PARA FIXAR

Filter seleciona: cada item precisa responder verdadeiro ou falso à condição.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Dado um array de números, use **filter** para retornar apenas os números pares.

■ Médio

Dado um array de objetos **produtos** (com **nome**, **preco**, **disponivel**), use **filter** para retornar apenas os produtos com **disponivel === true**.

■ Avançado

Combine **filter** + **map** em sequência: dado um array de objetos **alunos** (**nome**, **nota**), retorne um array só com os **nomes** dos alunos aprovados (**nota >= 6**). Primeiro filtre os aprovados, depois mapeie para pegar só os nomes.

Conceito: `find`

O que é?

Método de array que retorna o **primeiro elemento** que satisfaz uma condição (ou `undefined` se nenhum satisfizer) — diferente de `filter`, que retorna um array inteiro.

Para que serve?

Buscar um item específico, como "encontrar o produto com esse id" ou "encontrar o usuário com esse e-mail".

Sintaxe:

```
const encontrado = array.find((item) => item.propriedade === valor);
```

Exemplo:

```
const produtos = [
  { id: 1, nome: "Mouse" },
  { id: 2, nome: "Teclado" }
];
const produto = produtos.find((p) => p.id === 2);
console.log(produto); // { id: 2, nome: "Teclado" }
```

■ **Armadilha comum:** confundir `find` (retorna um item, ou `undefined`) com `filter` (retorna um array, mesmo que vazio []). Se você tentar usar `.nome` direto no resultado de um `find` que não encontrou nada, vai receber um erro (porque `undefined.nome` não existe).

■ PARA FIXAR

Find procura o primeiro item que satisfaz uma condição.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Dado um array de objetos `usuarios` (com `id` e `nome`), use `find` para encontrar o usuário com `id === 3` e imprima o nome.

■ Médio

Crie uma função `buscarProdutoPorNome(produtos, nome)` que usa `find` para retornar o produto cujo `nome` seja igual ao parâmetro recebido.

■ Avançado

Simule uma busca de "login": dado um array de **usuarios** com **email** e **senha**, crie uma função **autenticar(usuarios, email, senha)** que usa **find** para localizar o usuário com aquele e-mail, e depois verifica se a senha bate, retornando mensagens diferentes para "usuário não encontrado", "senha incorreta" e "login realizado com sucesso".

Conceito: **includes**

O que é?

Metodo disponível tanto em **strings** quanto em **arrays** que verifica se um valor está contido, retornando **true** ou **false**.

Para que serve?

Verificações rápidas de existência, muito usadas em buscas e filtros.

Sintaxe:

```
"JavaScript".includes("Script"); // true
[1, 2, 3].includes(2);           // true
```

Exemplo:

```
const tags = ["frontend", "javascript", "web"];
console.log(tags.includes("javascript")); // true
```

Muito comum combinar com **filter** para implementar busca em uma lista:

```
const nomes = ["Ana", "Bruno", "Carla"];
const termoBusca = "an";
const resultado = nomes.filter((nome) =>
  nome.toLowerCase().includes(termoBusca.toLowerCase())
);
console.log(resultado); // ["Ana"]
```

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Verifique com `includes` se a palavra "gato" está dentro do array `["cachorro", "gato", "peixe"]`.

■ Médio

Crie uma função `contemPalavra(texto, palavra)` que retorna `true/false` usando `.includes()`, ignorando maiúsculas/minúsculas (dica: `.toLowerCase()` nos dois lados).

■ Avançado

Dado um array de objetos `produtos` (com `nome`), implemente uma função `buscarProdutos(produtos, termo)` que usa `filter` + `includes` para retornar todos os produtos cujo nome contém o termo digitado — esse é exatamente o mecanismo por trás de qualquer campo de busca em um e-commerce.

■ Desafio de revisão — Módulo 4

Dado o array abaixo:

```
const produtos = [
  { nome: "Mouse", preco: 50, categoria: "periféricos", disponivel: true },
  { nome: "Teclado", preco: 120, categoria: "periféricos", disponivel: false },
  { nome: "Monitor", preco: 900, categoria: "telas", disponivel: true },
  { nome: "Cabo HDMI", preco: 30, categoria: "acessórios", disponivel: true }
];
```

1. Use `filter` para listar só os produtos disponíveis.
2. Use `map` para criar uma lista de textos no formato "Mouse - R\$ 50" só com os disponíveis.
3. Use `find` para localizar o produto "Monitor".
4. Use `filter` + `includes` para buscar produtos que contenham "cabo" no nome (ignorando maiúsculas/minúsculas).

Módulo 5 — Introdução ao DOM

Pré-requisito: Módulos 1 a 4. A partir daqui, todo exercício deve ser feito em arquivos reais (`index.html` + `style.css` + `script.js`), nunca só no console.

■ COMO ESTUDAR ESTE MÓDULO

Foco: Separe mentalmente linguagem JavaScript de recursos do navegador. Isso evita confusão entre JS e Web APIs.

Prática: Dica: quando aprender uma função, pergunte: 'isso pertence ao JavaScript ou ao navegador?'

Material: YouTube PT-BR: Curso em Vídeo — JavaScript para iniciantes e integração com páginas.

[Abrir material recomendado](#)

■ Antes de começar: diferenciando conceitos importantes

É muito comum misturar esses quatro conceitos — vale entender a diferença desde já:

Conceito	O que é
JavaScript (a linguagem)	A linguagem de programação em si: variáveis, funções, arrays, tudo que vimos até aqui. Funciona em qualquer lugar (navegador, servidor com Node.js, etc).
JavaScript no navegador	O ambiente onde o JS roda dentro do Chrome/Firefox, que disponibiliza recursos extras que não existem "na linguagem pura" (como o objeto <code>document</code>).
DOM (Document Object Model)	Uma representação em forma de "árvore de objetos" do seu HTML, que o navegador cria e que o JavaScript pode ler e modificar. Quando você faz <code>document.querySelector</code> , está navegando nessa árvore.
Web APIs	Funcionalidades que o navegador oferece ao JavaScript além do DOM: <code>fetch</code> (Módulo 9), <code>localStorage</code> (Módulo 10), temporizadores (<code>setTimeout</code>), geolocalização, etc. Não fazem parte da linguagem JavaScript — são fornecidas pelo ambiente.
APIs externas	Serviços de terceiros (em outro servidor) que você consome via <code>fetch</code> , geralmente retornando dados em JSON (Módulo 9). Não têm relação direta com o DOM.

Resumindo: **DOM é sobre a página**; **Web APIs** são ferramentas do navegador; **APIs externas** são serviços de fora.

Conceito: `document.querySelector()`

O que é?

Método que busca e retorna o **primeiro** elemento do HTML que corresponde a um seletor CSS.

Para que serve?

É a principal forma de "pegar" um elemento HTML para manipulá-lo com JavaScript — você usa exatamente a mesma sintaxe de seletor do CSS.

Sintaxe:

```
const elemento = document.querySelector(".classe");
const outro = document.querySelector("#id");
const terceiro = document.querySelector("button");
```

Exemplo:

```
<p class="titulo">Olá</p>
```

```
const titulo = document.querySelector(".titulo");
console.log(titulo); // <p class="titulo">Olá</p>
```

■ ■ **Armadilha comum:** se o `<script>` rodar **antes** do elemento existir no HTML (ex: `<script>` no `<head>` sem `defer`), `querySelector` retorna `null`, e tentar usar `.textContent` nesse `null` gera erro. Sempre confira a posição do `<script>` (ver seção de estrutura de projeto).

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um `index.html` com um `<h1 id="titulo">` e um `<p class="descricao">`. No `script.js`, selecione os dois com `querySelector` e imprima ambos no console.

■ Médio

Crie três `<div class="card">` no HTML. Use `querySelector(".card")` e observe que ele retorna **apenas o primeiro**. Anote em comentário por que isso acontece (você vai entender o motivo de existir o `querySelectorAll`).

■ Avançado

Monte uma pequena página com um formulário simples (`<input>`, `<button>`) e selecione ambos os elementos com `querySelector`, guardando em variáveis com nomes descritivos (`inputNome`, `botaoEnviar`). Você vai reaproveitar essa estrutura nos próximos módulos.

Conceito: `document.querySelectorAll()`

O que é?

Igual ao `querySelector`, mas retorna **todos** os elementos que casam com o seletor, em uma estrutura chamada `NodeList` (parecida com um array).

Para que serve?

Selecionar múltiplos elementos de uma vez para aplicar alguma ação a todos (ex: todos os itens de uma lista).

Sintaxe:

```
const cards = document.querySelectorAll(".card");
```

Exemplo:

```
const itens = document.querySelectorAll("li");
itens.forEach((item) => {
  console.log(item.textContent);
});
```

■ **Armadilha comum:** `NodeList` não é um array de verdade — não tem `.map()` ou `.filter()` em navegadores mais antigos, mas `.forEach()` funciona normalmente nos navegadores modernos. Se precisar usar `.map()/.filter()`, converta antes com `Array.from(nodeList)`.

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie uma lista `<ul>` com 5 `<li>`. Use `querySelectorAll("li")` e `forEach` para imprimir o texto de cada item no console.

■ Médio

Crie vários `<div class="produto">`, cada um com um preço diferente escrito dentro. Use `querySelectorAll` + `forEach` para imprimir só os textos que contêm "R\$".

■ Avançado

Com uma lista de `<li class="tarefa">` (algumas com uma classe extra `.concluida`), use `querySelectorAll(".tarefa")` e um `forEach` para imprimir no console apenas o texto das tarefas que **não** têm a classe `.concluida` (dica: `.classList.contains()`, adiantando o próximo tópico).

Conceito: `document.getElementById()`

O que é?

Forma "clássica" (anterior ao `querySelector`) de buscar um elemento pelo seu `id`.

Para que serve?

Mesmo objetivo do `querySelector("#id")`, mas é um pouco mais rápido e ainda muito usado em código legado e em exemplos por aí.

Sintaxe:

```
const elemento = document.getElementById("titulo"); // sem o #
```

Exemplo:

```
<h1 id="titulo">Olá</h1>
```

```
const titulo = document.getElementById("titulo");
```

Quando usar cada um: para projetos novos, prefira `querySelector/querySelectorAll` — são mais flexíveis porque aceitam qualquer seletor CSS (classe, tag, atributo, etc), não só id. `getElementById` ainda aparece bastante no mercado, então vale saber ler.

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Refaça o primeiro exercício de `querySelector` usando `getElementById` no lugar, para o elemento que tem `id`.

■ Médio

Crie um `<div id="resultado"></div>` vazio. Selecione-o com `getElementById` e, no próximo tópico (`textContent`), volte aqui para preenchê-lo dinamicamente.

■ Avançado

Explique em um comentário no código: em que situação `getElementById` **não** seria suficiente e você precisaria obrigatoriamente de `querySelector/querySelectorAll`? (pense em selecionar por classe, ou múltiplos elementos)

Conceito: `textContent`

O que é?

Propriedade que lê ou altera o **texto puro** dentro de um elemento HTML.

Para que serve?

É a forma mais segura de exibir texto dinâmico na tela (contador, mensagens, resultados).

Sintaxe:

```
const p = document.querySelector("p");
console.log(p.textContent); // lê o texto atual
p.textContent = "Novo texto"; // altera o texto
```

Exemplo:

```
<p id="mensagem">Carregando...</p>
```

```
const mensagem = document.querySelector("#mensagem");
mensagem.textContent = "Pronto!";
```

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um `<p id="saudacao">` vazio. No JS, defina o `textContent` dele como `"Bem-vindo!"`.

■ Médio

Crie um `<span id="contador">0</span>`. No JS, leia o valor atual (`.textContent`), converta para número, some 1, e atualize o `textContent` com o novo valor (isso é a base de um contador — você fará o botão clicável no Módulo 6).

■ Avançado

Crie 3 `<p class="preco">` com valores diferentes escritos como texto (ex: `"R$ 100"`). Use `querySelectorAll` + `forEach`, extraia o número de cada `textContent` (removendo o `"R$ "`), some tudo e

exiba o total em um `<p id="total">`.

Conceito: `innerHTML`

O que é?

Propriedade parecida com `textContent`, mas que lê/escreve o conteúdo interpretando **tags HTML**, não só texto puro.

Para que serve?

Inserir HTML formatado dinamicamente (ex: gerar vários `<li>` de uma vez a partir de um array).

Sintaxe:

```
const div = document.querySelector("#container");
div.innerHTML = "<strong>Texto em negrito</strong>";
```

Exemplo:

```
const lista = document.querySelector("#lista");
const frutas = ["Maçã", "Banana", "Uva"];
lista.innerHTML = frutas.map(fruta => `<li>${fruta}</li>`).join("");
```

■ **Armadilha comum e importante — segurança:** `innerHTML` interpreta **qualquer** HTML, inclusive `<script>` maliciosos, se o conteúdo vier de uma fonte não confiável (ex: texto digitado por outro usuário). Isso é chamado de vulnerabilidade **XSS**. Para exibir texto simples, `textContent` é sempre mais seguro. Use `innerHTML` quando você realmente precisa inserir tags, e com dados que você controla ou já validou.

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um `<div id="caixa">` vazio. Use `innerHTML` para inserir "`<em>Texto em itálico</em>`".

■ Médio

Dado um array de nomes, use `.map()` (Módulo 4!) + `.join("")` para montar uma string com vários `<li>` e insira tudo de uma vez em uma `<ul id="lista">` usando `innerHTML`.

■ Avançado

Dado um array de objetos `produtos (nome, preco)`, gere dinamicamente, usando `innerHTML`, um card para cada produto dentro de uma `<div id="catalogo">`, no formato:

```
<div class="produto"><h3>Mouse</h3><p>R$ 50</p></div>
```

Combine `map` + template literals + `join("")`.

Conceito: value

O que é?

Propriedade específica de elementos de formulário (`<input>`, `<select>`, `<textarea>`) que lê ou define o valor digitado/selecionado.

Para que serve?

Ler o que o usuário digitou, ou preencher um campo programaticamente.

Sintaxe:

```
const input = document.querySelector("#nome");
console.log(input.value); // lê o que foi digitado
input.value = "Rafael"; // preenche o campo
```

Exemplo:

```
<input type="text" id="nomeUsuario">
```

```
const input = document.querySelector("#nomeUsuario");
console.log(`Você digitou: ${input.value}`);
```

■ **Armadilha comum:** `.value` de um `<input type="number">` ainda retorna uma **string** — sempre converta com `Number()` antes de fazer contas.

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um `<input id="nome">`. No console do navegador (com a página aberta), leia o `.value` depois de digitar algo manualmente no campo.

■ Médio

Crie um `<input type="number" id="idade">` e um `<p id="resultado">`. No JS, leia o `value`, converta para número, e escreva em `resultado` se a pessoa é maior ou menor de idade (você ainda vai fazer isso reagir a um clique no Módulo 6 — por enquanto rode o código manualmente após preencher o campo).

■ Avançado

Crie dois `<input type="number">` (`valorA`, `valorB`) e um `<select id="operacao">` com opções `+`, `-`, `*`, `/`. Leia os três valores e calcule o resultado usando um `if/else` ou `switch`, exibindo em um `<p id="resultado">`.

Conceito: `document.createElement()`

O que é?

Metodo que cria um novo elemento HTML "em memória", que ainda não está visível na página até ser inserido.

Para que serve?

Criar elementos dinamicamente de forma mais controlada e segura que `innerHTML` (evita problemas de XSS e permite adicionar eventos ao elemento antes de inserir).

Sintaxe:

```
const novoParagrafo = document.createElement("p");
novoParagrafo.textContent = "Criado via JS";
```

Exemplo:

```
const li = document.createElement("li");
li.textContent = "Nova tarefa";
li.classList.add("tarefa");
```

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Use `createElement` para criar um `<p>` com o texto "**Criado dinamicamente**" (ainda sem inserir na página — isso vem no próximo tópico).

■ Médio

Crie 3 elementos `<li>` com `createElement`, cada um com um texto diferente, guardando-os em um array.

■ Avançado

Compare mentalmente (em um comentário no código) as vantagens e desvantagens de usar `createElement` versus `innerHTML` para gerar uma lista de 20 itens. Em qual situação cada abordagem faz mais sentido?

Conceito: `appendChild` e formas modernas de inserção

O que é?

Métodos para efetivamente **inserir** um elemento (criado com `createElement`) dentro de outro elemento já existente na página.

Para que serve?

Completar o processo de criação dinâmica de elementos: criar → configurar → inserir.

Sintaxe:

```
const lista = document.querySelector("#lista");
const li = document.createElement("li");
li.textContent = "Nova tarefa";

lista.appendChild(li);          // forma clássica
// ou, forma moderna equivalente:
lista.append(li);               // aceita também texto puro e múltiplos itens
```

Formas modernas úteis:

```
lista.append(li);               // insere no final
lista.prepend(li);             // insere no início
elemento.insertAdjacentHTML("beforeend", "<li>Item</li>"); // insere HTML direto
elemento.remove();              // remove o elemento da página
```

Exemplo completo:

```
const lista = document.querySelector("#lista");

const novaTarefa = document.createElement("li");
novaTarefa.textContent = "Estudar DOM";

lista.appendChild(novaTarefa);
```

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie uma `<ul id="lista">` vazia no HTML. No JS, crie um `<li>` com `createElement`, defina o texto e insira na lista com `appendChild`.

■ Médio

Dado um array de 5 nomes, use um loop (`forEach`) para criar um `<li>` para cada nome e inserir todos na lista, um por um.

■ Avançado

Crie uma `<div id="catalogo">` vazia. Dado um array de objetos `produtos` (`nome`, `preco`), para cada produto: crie uma `<div class="card">`, dentro dela crie um `<h3>` (nome) e um `<p>` (preço) com `createElement`, monte a estrutura (`appendChild` dentro do card) e insira o card no catálogo. Compare o resultado visual com o exercício avançado de `innerHTML` — o resultado deve ser o mesmo, mudando só a técnica.

Conceito: `classList`

O que é?

Propriedade que dá acesso às classes CSS de um elemento, permitindo adicionar, remover, alternar ou verificar classes.

Para que serve?

É a forma correta de alterar a **aparência** de um elemento via JavaScript — em vez de mexer em **style** diretamente, você adiciona/remove classes CSS já definidas no **style.css**, mantendo a separação entre lógica (JS) e visual (CSS).

Sintaxe:

```
elemento.classList.add("ativo");
elemento.classList.remove("ativo");
elemento.classList.toggle("ativo"); // adiciona se não tem, remove se tem
elemento.classList.contains("ativo"); // true/false
```

Exemplo:

```
.destaque {
  background-color: yellow;
  font-weight: bold;
}
```

```
const titulo = document.querySelector("#titulo");
titulo.classList.add("destaque");
```

■ **Armadilha comum:** alterar estilos diretamente via `elemento.style.backgroundColor = "yellow"` funciona, mas mistura lógica com estilo e é mais difícil de manter. **Prefira sempre `classList`**, deixando o CSS decidir a aparência e o JS decidir apenas "quando" aplicar aquela aparência.

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

No CSS, crie uma classe `.destaque` com alguma cor de fundo diferente. No JS, selecione um `<p>` e adicione essa classe com `classList.add`.

■ Médio

Crie 3 `<div class="card">`. No JS, use `querySelectorAll` + `forEach` para adicionar a classe `.destaque` apenas na segunda `<div>` (dica: use o índice do `forEach`).

■ Avançado

Crie uma `<div id="caixa">` e um botão (ainda sem funcionalidade de clique, isso vem no Módulo 6). No CSS, crie duas classes, `.tema-claro` e `.tema-escuro`, com estilos bem diferentes. No JS, use `classList.toggle` para alternar entre os dois temas na `<div>`, executando o toggle diretamente no carregamento do script (depois, no Módulo 6, você vai conectar isso a um clique de verdade).

■ Desafio de revisão — Módulo 5

Crie uma página (`index.html` + `style.css` + `script.js`) com:

1. Uma `<div id="lista-tarefas">` vazia.
2. Um array de tarefas no JS: `["Estudar DOM", "Praticar exercícios", "Revisar módulo"]`.
3. Para cada tarefa, crie dinamicamente (`createElement` ou `innerHTML`, sua escolha) um elemento mostrando o texto da tarefa, com uma classe `.tarefa` definida no CSS (com algum estilo visual, como borda ou padding).
4. Adicione a classe `.concluida` (com CSS diferenciado, ex: texto riscado) na **segunda** tarefa da lista, usando `classList`.

Não é necessário clique ainda — isso vem no próximo módulo.

Módulo 6 — Eventos

*Pré-requisito: Módulo 5. Este módulo é o que faz sua página **reagir** ao usuário — a partir daqui suas páginas deixam de ser estáticas.*

■ COMO ESTUDAR ESTE MÓDULO

Foco: Priorize funções, arrays e objetos como ferramentas para organizar dados. Esses conceitos serão usados diretamente no DOM e nas APIs.

Prática: Dica: transforme exercícios pequenos em funções reutilizáveis.

Material: YouTube PT-BR: Curso em Vídeo — funções, variáveis compostas e desafios.

[Abrir material recomendado](#)

Conceito: `addEventListener`

O que é?

Metodo que "escuta" uma ação (evento) em um elemento e executa uma função quando ela acontece.

Para que serve?

É a forma padrão de conectar interações do usuário (clique, digitação, envio de formulário) a código JavaScript.

Sintaxe:

```
elemento.addEventListener("nomeDoEvento", funcao);
```

Exemplo:

```
const botao = document.querySelector("#meuBotao");

botao.addEventListener("click", () => {
  console.log("Botão clicado!");
});
```

■ **Armadilha comum:** escrever `botao.addEventListener("click", minhaFuncao())` com **parênteses** executa a função imediatamente ao carregar a página (e passa o *resultado* dela como segundo argumento) em vez de esperar o clique. O correto é passar a função **sem chamar**: `minhaFuncao` (sem `()`), ou uma arrow function que chama por dentro: `() => minhaFuncao()`.

■ PARA FIXAR

Evento é um acontecimento; `addEventListener` é uma forma de dizer o que fazer quando ele acontecer.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um botão `<button id="btn">Clique aqui</button>`. Adicione um `addEventListener` de `click` que mostra `"Cliquei!"` no console.

■ Médio

Crie um botão e um `<p id="mensagem">`. Ao clicar, altere o `textContent` do parágrafo para `"Você clicou!"`.

■ Avançado

Crie um contador completo: um `<span id="contador">0</span>`, um botão `"+1"` e um botão `"-1"`. Cada botão deve ter seu próprio `addEventListener`, lendo o valor atual do `textContent`, somando/subtraindo 1, e atualizando a tela. Trate o caso do contador ficar negativo, impedindo (com `if`) que ele passe de 0 para baixo.

Conceito: `event` e `event.preventDefault()`

O que é?

Toda função chamada por um evento recebe automaticamente um objeto `event` (também chamado `e`), que carrega informações sobre o que aconteceu (qual tecla, qual elemento, etc). `event.preventDefault()` é um

método desse objeto que **cancela o comportamento padrão** do navegador para aquele evento.

Para que serve?

event permite reagir de forma mais específica ao que aconteceu. `preventDefault()` é essencial em formulários, porque o comportamento padrão de um **submit** é **recarregar a página** — o que normalmente não é o que você quer em uma aplicação JavaScript.

Sintaxe:

```
elemento.addEventListener("click", (event) => {
  console.log(event.target); // o elemento que foi clicado
});

formulario.addEventListener("submit", (event) => {
  event.preventDefault(); // impede o recarregamento da página
});
```

Exemplo:

```
<button id="btn" data-id="42">Clique</button>
```

```
const botao = document.querySelector("#btn");
botao.addEventListener("click", (event) => {
  console.log(event.target.dataset.id); // "42"
});
```

■ PARA FIXAR

Evento é um acontecimento; `addEventListener` é uma forma de dizer o que fazer quando ele acontecer.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um botão e, no `addEventListener`, imprima `event.type` no console (deve mostrar `"click"`).

■ Médio

Crie 3 botões diferentes que compartilham a **mesma** função de clique. Dentro da função, use `event.target.textContent` para identificar qual botão foi clicado e imprimir uma mensagem diferente para cada um.

■ Avançado

Crie um `<form>` simples com um `<input>` e um botão `type="submit"`. Adicione um `addEventListener("submit", ...)` no formulário, chame `event.preventDefault()` logo no início da função, e imprima no console o valor digitado no input. Confirme que a página **não recarrega** ao enviar.

Conceito: Evento `input`

O que é?

Evento disparado **a cada mudança** de valor em um campo de texto (a cada tecla digitada, colada ou removida).

Para que serve?

Reagir em tempo real ao que o usuário está digitando — base de qualquer busca "ao vivo" ou validação instantânea.

Sintaxe:

```
input.addEventListener("input", (event) => {
  console.log(event.target.value);
});
```

Exemplo:

```
<input type="text" id="busca">
<p id="resultado"></p>
```

```
const busca = document.querySelector("#busca");
const resultado = document.querySelector("#resultado");

busca.addEventListener("input", (event) => {
  resultado.textContent = `Você digitou: ${event.target.value}`;
});
```

■ PARA FIXAR

Evento é um acontecimento; `addEventListener` é uma forma de dizer o que fazer quando ele acontecer.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um `<input>` e um `<p>`. A cada tecla digitada, mostre o valor atual do input dentro do parágrafo.

■ Médio

Crie um `<input>` e mostre, em tempo real, quantos caracteres foram digitados (`.length` do value).

■ Avançado — pré-requisito: Módulo 4 (`filter`, `includes`)

Crie um `<input id="busca">` e uma lista de nomes já renderizada na tela (`<ul id="lista">`, gerada a partir de um array com `map/createElement`, como no Módulo 5). A cada `input`, filtre o array original (`filter` + `includes`, ignorando maiúsculas/minúsculas) e **re-renderize** a lista mostrando só os nomes que combinam com o texto digitado. Esse é o mecanismo por trás de qualquer campo de busca com filtro em tempo real.

Conceito: Evento `change`

O que é?

Similar ao `input`, mas disparado quando o valor é alterado e o elemento perde o foco (ou, em `<select>` e checkboxes, assim que a seleção muda).

Para que serve?

Ideal para `<select>`, `<input type="checkbox">` e `<input type="radio">`, onde não faz sentido reagir a cada tecla — faz mais sentido reagir quando a escolha é finalizada.

Sintaxe:

```
select.addEventListener("change", (event) => {
  console.log(event.target.value);
});
```

Exemplo:

```
<select id="cor">
  <option value="vermelho">Vermelho</option>
  <option value="azul">Azul</option>
</select>
```

```
const select = document.querySelector("#cor");
select.addEventListener("change", (event) => {
  document.body.style.backgroundColor = event.target.value;
});
```

input vs change — quando usar cada um: use `input` para reagir enquanto o usuário digita em campos de texto; use `change` para `<select>`, checkbox, radio, ou quando você só se importa com o valor **final**, não com cada tecla.

■ PARA FIXAR

Evento é um acontecimento; `addEventListener` é uma forma de dizer o que fazer quando ele acontecer.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um `<select>` com 3 opções de cor. Ao mudar a seleção, imprima o valor escolhido no console.

■ Médio

Crie um `<input type="checkbox" id="modoEscuro">`. Ao marcar/desmarcar, use `classList.toggle` (Módulo 5) no `<body>` para alternar entre um tema claro e escuro definidos no CSS.

■ Avançado — pré-requisito: Módulo 4 (`filter`)

Crie um `<select id="filtroCategoria">` com opções de categorias (ex: "Todos", "Eletrônicos", "Roupas") e uma lista de produtos já renderizada (como no exercício avançado de `input`). Ao mudar a seleção, filtre o array original por categoria e re-renderize a lista.

Conceito: Evento `submit`

O que é?

Evento disparado quando um `<form>` é enviado (seja clicando no botão `type="submit"` ou apertando Enter dentro de um campo do formulário).

Para que serve?

Capturar todos os dados de um formulário de uma vez, no momento correto — sempre combinado com `event.preventDefault()` em aplicações JavaScript.

Sintaxe:

```
formulario.addEventListener("submit", (event) => {
  event.preventDefault();
  // ler valores dos inputs aqui
});
```

Exemplo:

```
<form id="formTarefa">
  <input type="text" id="tarefa" placeholder="Nova tarefa">
  <button type="submit">Adicionar</button>
</form>
```

```
const form = document.querySelector("#formTarefa");
const inputTarefa = document.querySelector("#tarefa");

form.addEventListener("submit", (event) => {
  event.preventDefault();
  console.log(`Nova tarefa: ${inputTarefa.value}`);
  inputTarefa.value = ""; // limpa o campo após enviar
});
```

■ **Armadilha comum:** colocar o `addEventListener` no **botão** com evento `click` em vez de no `<form>` com `submit` funciona parcialmente, mas perde o comportamento de "enviar com Enter" e é considerado menos correto semanticamente. Prefira sempre escutar o `submit` do formulário.

■ PARA FIXAR

Evento é um acontecimento; `addEventListener` é uma forma de dizer o que fazer quando ele acontecer.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um `<form>` com um `<input>` e um botão de envio. Ao submeter, use `preventDefault()` e imprima o valor digitado no console.

■ Médio

No mesmo formulário, depois de capturar o valor, limpe o campo (`input.value = ""`) e mostre uma mensagem de confirmação em um `<p>`.

■ Avançado — combina tudo do módulo

Crie um formulário com `<input>` para "nome da tarefa" e um botão de envio. Ao submeter: previna o comportamento padrão, valide que o campo não está vazio (se estiver, mostre uma mensagem de erro e **não** continue), crie um novo elemento na lista de tarefas na tela (Módulo 5) com o texto digitado, e limpe o input. Esse exercício é a base do Projeto 1 (lista de tarefas).

■ Desafio de revisão — Módulo 6

Construa uma pequena calculadora de IMC:

- Dois `<input type="number">`: peso (kg) e altura (m).

- Um `<form>` envolvendo os campos e um botão "Calcular".
- Ao submeter: previna o recarregamento, valide que os dois campos foram preenchidos (se não, mostre erro), calcule o IMC (`peso / (altura * altura)`), e exiba o resultado em um `<p>`, junto com uma classificação ("Abaixo do peso", "Peso normal", "Sobrepeso", etc) usando `if/else`.

Módulo 7 — Formulários e manipulação dinâmica avançada

Pré-requisito: Módulos 5 e 6. Aqui juntamos tudo (DOM + eventos + arrays/objetos) para construir interfaces completas.

■ COMO ESTUDAR ESTE MÓDULO

Foco: O objetivo é conseguir fazer a página reagir ao usuário. Pratique input, click, change e submit separadamente antes de combiná-los.

Prática: Dica: faça mini interfaces: contador, formulário, filtro e troca de tema.

Material: YouTube PT-BR: Curso em Vídeo — DOM e eventos; complemento com projetos próprios.

[Abrir material recomendado](#)

Conceito: Formulários

O que é?

Uma revisão de como HTML, CSS e JS trabalham juntos em um `<form>`: os diferentes tipos de campo (`text`, `number`, `email`, `checkbox`, `radio`, `select`) e como cada um se comporta ao ser lido via JavaScript.

Para que serve?

É a principal forma de coletar dados do usuário em qualquer aplicação web.

Sintaxe — lendo diferentes tipos de campo:

```
inputTexto.value;           // string
inputNumero.value;         // string (converta com Number())
checkbox.checked;             // boolean (true/false)
radioSelecionado.value;     // valor do radio marcado
select.value;              // valor da opção selecionada
```

Exemplo — formulário com vários tipos de campo:

```
<form id="formCadastro">
  <input type="text" id="nome" placeholder="Nome">
  <input type="number" id="idade" placeholder="Idade">
  <input type="checkbox" id="aceitaTermos"> Aceito os termos
  <button type="submit">Cadastrar</button>
</form>
```

```
const form = document.querySelector("#formCadastro");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  const nome = document.querySelector("#nome").value;
  const idade = Number(document.querySelector("#idade").value);
  const aceitou = document.querySelector("#aceitaTermos").checked;

  console.log({ nome, idade, aceitou });
});
```

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um formulário com um `<input type="text">` e um `<input type="checkbox">`. Ao submeter, imprima os dois valores no console (`value` e `checked`).

■ Médio

Adicione validação: o cadastro só deve prosseguir (imprimir os dados) se o nome não estiver vazio e o checkbox de termos estiver marcado. Caso contrário, mostre uma mensagem de erro em um `<p>`.

■ Avançado

Monte um formulário de cadastro de produto (`nome`, `preco`, `categoria` via `<select>`). Ao submeter, monte um objeto `{ nome, preco, categoria }` com os dados (convertendo `preco` para número) e adicione esse objeto a um array `produtosCadastrados`. Renderize a lista de produtos cadastrados na tela toda vez que um novo for adicionado.

Conceito: Manipulação de classes CSS pelo JavaScript (aprofundando)

O que é?

Retomando o `classList` do Módulo 5, agora combinado com eventos para criar interações visuais reais: menus, modais, alternância de estados.

Para que serve?

É assim que praticamente toda interface interativa (menu mobile, abas, modais, temas) é construída sem frameworks.

Exemplo — menu que abre/fecha:

```
<button id="btnMenu">■ Menu</button>
<nav id="menu" class="menu-fechado">
  <a href="#">Início</a>
  <a href="#">Sobre</a>
</nav>
```

```
.menu-fechado { display: none; }
.menu-aberto { display: block; }
```

```
const btnMenu = document.querySelector("#btnMenu");
const menu = document.querySelector("#menu");

btnMenu.addEventListener("click", () => {
  menu.classList.toggle("menu-aberto");
  menu.classList.toggle("menu-fechado");
});
```

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um botão que, ao ser clicado, adiciona a classe `.destaque` (já estilizada no CSS) em um `<p>`.

■ Médio

Crie um menu de navegação escondido por padrão (`display: none` via classe). Um botão deve alternar (`classList.toggle`) entre mostrar e esconder o menu.

■ Avançado

Construa um modal (janela sobreposta) simples: uma `<div class="modal oculto">` com algum conteúdo e um botão de fechar (X) dentro dela, mais um botão "Abrir modal" fora. Use `classList` para mostrar/esconder o modal. Adicione também: fechar o modal ao clicar fora dele (dica: `event.target` no clique

de um overlay que cobre a tela toda).

Conceito: Criação dinâmica de elementos HTML (aprofundando)

O que é?

Consolidação do que foi visto no Módulo 5 (`createElement/innerHTML`), agora sempre disparada por eventos e trabalhando com **arrays de dados que mudam** ao longo do uso da página (adicionar, remover, atualizar itens).

Para que serve?

É o padrão central de qualquer aplicação frontend: manter uma "fonte de dados" (um array) e **re-renderizar** a interface toda vez que os dados mudam.

Exemplo — padrão "dados → renderizar":

```
let tarefas = ["Estudar", "Praticar"];
const lista = document.querySelector("#lista");

function renderizarLista() {
  lista.innerHTML = tarefas
    .map((tarefa) => `<li>${tarefa}</li>`)
    .join("");
}

renderizarLista(); // primeira renderização

// sempre que "tarefas" mudar, chame renderizarLista() de novo
tarefas.push("Revisar");
renderizarLista();
```

Esse padrão — **uma função que redesenha a tela inteira a partir dos dados atuais** — é a base de como frameworks como React funcionam por trás dos panos, só que de forma manual.

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Implemente o padrão do exemplo acima: um array de tarefas e uma função `renderizarLista()` que redesenha a `<ul>` a partir do array.

■ Médio

Adicione um botão "Adicionar tarefa aleatória" que, ao ser clicado, insere um novo item fixo no array (ex: "Nova tarefa") e chama `renderizarLista()` novamente.

■ Avançado

Adicione, para cada `<li>` renderizado, um botão "Remover" (crie-o dentro do próprio template/elemento). Ao clicar em remover, o item correspondente deve sair do array (dica: `filter`, criando um novo array sem aquele item) e a lista deve ser re-renderizada. Você vai precisar de alguma forma de identificar **qual** item remover — um `data-index` no elemento é uma boa estratégia.

■ Desafio de revisão — Módulo 7

Construa uma "lista de tarefas" funcional (protótipo do Projeto 1):

- Formulário com input + botão para adicionar tarefa.
- Array de tarefas em memória (cada tarefa é um objeto `{ texto, concluida }`).
- Cada tarefa renderizada na tela com um checkbox (marca como concluída, alternando classe CSS de "riscado") e um botão de remover.
- Toda mudança nos dados deve disparar a re-renderização da lista completa (padrão "dados → renderizar").

Ainda não é necessário salvar os dados permanentemente — isso vem no Módulo 10 (`localStorage`).

Módulo 8 — Assincronismo

Pré-requisito: Módulos 1 a 7. Este módulo prepara o terreno para consumir APIs (Módulo 9).

■ COMO ESTUDAR ESTE MÓDULO

Foco: Assíncrono é uma mudança de mentalidade: a operação pode terminar depois. Primeiro entenda Promise; depois use `async/await` para escrever de forma legível.

Prática: Dica: desenhe no papel a sequência 'começa → espera → termina → continua'.

Material: YouTube PT-BR: procure conteúdos sobre Promise, `async/await` e Event Loop; uma referência complementar em português é o material da Fernanda Kipper.

[Abrir material recomendado](#)

■ Por que assincronismo existe?

Até agora, todo código que escrevemos roda de forma **síncrona**: uma linha espera a anterior terminar. Mas algumas operações demoram um tempo imprevisível — como buscar dados na internet — e o navegador **não pode travar** enquanto espera (senão a página inteira congelaria). Por isso JavaScript tem mecanismos para lidar com código que "vai terminar em algum momento no futuro".

Conceito: Funções assíncronas (introdução)

O que é?

Uma função assíncrona é uma função que pode "pausar" sua execução para esperar uma operação demorada terminar, sem travar o restante da página.

Para que serve?

Buscar dados de uma API, ler arquivos, esperar temporizadores — qualquer coisa que não termina instantaneamente.

Exemplo conceitual (sem código ainda): imagine pedir uma pizza por telefone. Você não fica parado em silêncio esperando — você continua fazendo outras coisas, e quando a pizza chega (o "evento"), você reage a isso. É basicamente essa a ideia por trás do código assíncrono.

■ PARA FIXAR

Pense em função como uma pequena tarefa: recebe algo, processa e pode devolver algo.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Pesquise e explique, em um comentário no código, a diferença entre código **síncrono** e **assíncrono**, com um exemplo do cotidiano (fora de programação).

■ Médio

Use `setTimeout(() => { ... }, 2000)` (uma Web API, não faz parte da linguagem JS pura) para imprimir uma mensagem no console 2 segundos depois de rodar o script. Observe que o código **depois** do `setTimeout` roda antes da mensagem aparecer.

■ Avançado

Encadeie três `setTimeout` para simular um processo em etapas (ex: "**Carregando...**", depois de 1s "**Processando...**", depois de mais 1s "**Concluído!**"). Note como o código começa a ficar difícil de ler

conforme aninha mais chamadas — esse problema é resolvido pelas **Promises**, a seguir.

Conceito: Promise

O que é?

Um objeto que representa um valor que **ainda não existe**, mas vai existir (ou vai falhar) em algum momento futuro. Uma Promise tem 3 estados possíveis: **pending** (pendente), **fulfilled** (resolvida com sucesso) e **rejected** (rejeitada, com erro).

Para que serve?

É a base sobre a qual **fetch** (Módulo 9) e **async/await** funcionam. Você vai usar Promises quase sempre através do **async/await**, mas é importante entender o que está por trás.

Sintaxe:

```
const minhaPromise = new Promise((resolve, reject) => {
  const sucesso = true;

  if (sucesso) {
    resolve("Deu certo!");
  } else {
    reject("Deu errado!");
  }
});

minhaPromise
  .then((resultado) => console.log(resultado))
  .catch((erro) => console.log(erro));
```

Exemplo:

```
function esperar(segundos) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`Esperei ${segundos}s`), segundos * 1000);
  });
}

esperar(2).then((mensagem) => console.log(mensagem));
```

■ PARA FIXAR

Uma Promise representa uma operação cujo resultado ainda não está disponível.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie uma **Promise** simples que sempre resolve com o texto "**Pronto!**", e use **.then()** para imprimir esse texto no console.

■ Médio

Crie uma função **sortear()** que retorna uma **Promise**: sorteie um número aleatório entre 0 e 1, se for maior que 0.5 chame **resolve("Ganhou!")**, senão **reject("Perdeu!")**. Use **.then()** e **.catch()** para tratar os dois casos.

■ Avançado

Reescreva a função **esperar(segundos)** do exemplo acima e encadeie três chamadas usando **.then()** em sequência (cada uma esperando 1 segundo e imprimindo uma mensagem diferente), sem usar **async/await** ainda.

Conceito: **async** e **await**

O que é?

Sintaxe moderna que torna o trabalho com Promises muito mais legível — permite escrever código assíncrono "como se fosse síncrono", de cima para baixo, sem encadear vários **.then()**.

Para que serve?

É a forma padrão, no JavaScript atual, de lidar com qualquer operação assíncrona — especialmente **fetch** (Módulo 9).

Sintaxe:

```
async function minhaFuncao() {  
  const resultado = await algumaPromise();  
  console.log(resultado);  
}
```

- **async** antes de **function** indica que essa função vai trabalhar com código assíncrono e sempre retorna uma **Promise**.
- **await** só pode ser usado **dentro** de uma função **async**, e "pausa" a execução daquela função até a **Promise** ser resolvida, sem travar o restante da página.

Exemplo — reescrevendo o exemplo anterior:

```
function esperar(segundos) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`Esperei ${segundos}s`), segundos * 1000);
  });
}

async function executar() {
  const resultado1 = await esperar(1);
  console.log(resultado1);

  const resultado2 = await esperar(1);
  console.log(resultado2);
}

executar();
```

Compare com a versão de `.then()` encadeado do exercício anterior — `async/await` costuma ser bem mais fácil de ler.

■ PARA FIXAR

`async/await` organiza código assíncrono em uma leitura mais linear.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Reescreva o exercício médio de `Promise` (função `sortear()`) usando `async/await` em vez de `.then().catch()` (você vai precisar de `try/catch`, próximo tópico, para tratar o `reject`).

■ Médio

Crie uma função `async` que "carrega" 3 mensagens em sequência, cada uma esperando 1 segundo (reproveite a função `esperar`), imprimindo cada mensagem assim que estiver disponível.

■ Avançado

Simule o carregamento de uma página: crie uma função `async` `carregarPagina()` que atualiza o `textContent` de um `<p id="status">` para "Carregando dados do usuário...", espera 1 segundo (`await esperar(1)`), depois "Carregando pedidos...", espera mais 1 segundo, e por fim "Pronto!". Execute a função ao carregar a página.

Conceito: try / catch

O que é?

Estrutura para capturar e tratar erros que acontecem durante a execução do código, evitando que o script pare de funcionar completamente ou que a página quebre.

Para que serve?

Essencial em código assíncrono: uma requisição pode falhar (sem internet, servidor fora do ar, dados inválidos) e você precisa lidar com isso mostrando uma mensagem adequada ao usuário, em vez de deixar a aplicação travada ou silenciosamente quebrada.

Sintaxe:

```
try {  
  // código que pode dar erro  
} catch (erro) {  
  // executado se algo no try falhar  
  console.log(erro.message);  
}
```

Exemplo combinando com async/await:

```
async function buscarDados() {  
  try {  
    const resultado = await sortear(); // pode rejeitar  
    console.log(resultado);  
  } catch (erro) {  
    console.log(`Algo deu errado: ${erro}`);  
  }  
}
```

■ PARA FIXAR

Pense em try como 'tente executar' e catch como 'se der erro, trate aqui'.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Force um erro proposital (ex: chamar `.toUpperCase()` em uma variável `undefined`) dentro de um `try/catch` e imprima uma mensagem amigável no `catch`, em vez de deixar o erro "quebrar" o script.

■ Médio

Reescreva a função `sortear()` (do tópico de Promise) usando `async/await` dentro de um `try/catch`, tratando tanto o caso de sucesso quanto o de erro.

■ Avançado

Crie uma função `async carregarDados()` que tem 50% de chance de "falhar" (simule com `Math.random()` e `throw new Error("Falha ao carregar")`). Ao chamar essa função dentro de um `try/catch`, exiba na tela (em um `<p id="status">`) `"Dados carregados com sucesso"` ou `"Erro: não foi possível carregar os dados"`, conforme o resultado. Esse padrão é exatamente o que você vai usar ao consumir APIs de verdade no próximo módulo.

■ Desafio de revisão — Módulo 8

Crie uma função `async processarPedido()` que simula o processamento de um pedido em uma loja:

1. Mostra `"Verificando estoque..."` na tela, espera 1 segundo.
2. Sorteia (50/50) se o produto está em estoque; se não estiver, lança um erro (`throw new Error("Produto fora de estoque")`).
3. Se está em estoque, mostra `"Processando pagamento..."`, espera mais 1 segundo, depois `"Pedido confirmado!"`.
4. Toda a função deve estar dentro de um `try/catch`, mostrando uma mensagem de erro amigável na tela caso algo falhe.

Módulo 9 — Fetch, JSON e consumo de APIs

Pré-requisito: Módulos 1 a 8 (especialmente `async/await`, `try/catch`, `map/filter`, e criação dinâmica de elementos).

■ COMO ESTUDAR ESTE MÓDULO

Foco: Este é um dos módulos mais importantes para frontend. Memorize o fluxo: `fetch` → `Response` → `response.json()` → `dados` → `map/filter` → `DOM`.

Prática: Dica: antes de criar a interface, teste a API no console e descubra exatamente o formato do JSON.

Material: YouTube PT-BR: CFB Cursos — aula sobre FETCH para consumo de APIs em JavaScript.

[Abrir material recomendado](#)

■ O fluxo completo de uma requisição

Antes dos detalhes, entenda o caminho que os dados percorrem — isso é a espinha dorsal deste módulo:

```

fetch(url)
↓ (faz a requisição HTTP para o endereço)
Response
↓ (objeto de resposta "bruta" do servidor — ainda não é o dado em si)
response.json()
↓ (converte o corpo da resposta, que veio como texto JSON, em objeto/array JS)
dados
↓ (agora é um array ou objeto JavaScript normal, igual aos que você já manipula)
arrays/objetos
↓
map / filter / forEach
↓ (transforma os dados no formato que você quer exibir)
DOM
↓ (createElement / innerHTML para colocar na tela)
interface

```

Cada seta acima é uma etapa que você já sabe fazer isoladamente — este módulo só conecta tudo.

Conceito: `fetch`

O que é?

Uma **Web API** do navegador (não faz parte da linguagem JS — lembra da tabela do Módulo 5?) que faz requisições HTTP para buscar (ou enviar) dados de um servidor.

Para que serve?

É a forma padrão de "conversar" com uma API pela internet a partir do navegador.

Sintaxe:

```

fetch("https://alguma-api.com/dados")
  .then((response) => response.json())
  .then((dados) => console.log(dados))
  .catch((erro) => console.log(erro));

```

Forma equivalente com `async/await` (recomendada, mais legível):

```

async function buscarDados() {
  try {
    const response = await fetch("https://alguma-api.com/dados");
    const dados = await response.json();
    console.log(dados);
  } catch (erro) {
    console.log("Erro ao buscar dados:", erro);
  }
}

```

Exemplo com uma API pública e gratuita, sem necessidade de chave ([PokéAPI](#)):

```

async function buscarPokemon(nome) {
  try {
    const response = await fetch(`https://pokeapi.co/api/v2/pokemon/${nome}`);
    const dados = await response.json();
    console.log(dados.name, dados.height, dados.weight);
  } catch (erro) {
    console.log("Pokémon não encontrado");
  }
}

buscarPokemon("pikachu");

```

■ ■ **Armadilha comum:** `fetch` não rejeita a `Promise` (não cai no `catch`) quando o servidor responde com erro HTTP (como 404). Ele só rejeita em falhas de rede (sem internet, URL inválida). Por isso é comum verificar `response.ok` manualmente:

```
if (!response.ok) {  
  throw new Error(`Erro ${response.status}`);  
}
```

■ PARA FIXAR

Separe mentalmente requisição, resposta e dados: `fetch` não entrega o JSON pronto.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Use `fetch` com `async/await` para buscar dados de <https://pokeapi.co/api/v2/pokemon/pikachu> e imprima o objeto completo no console.

■ Médio

Crie uma função `buscarPokemon(nome)` reutilizável (como no exemplo) e chame-a para 3 pokémons diferentes, imprimindo o nome e o tipo de cada um (dica: `dados.types`, que é um array de objetos).

■ Avançado

Adicione tratamento de erro: se o nome do pokémon não existir, a API retorna 404. Use `response.ok` para detectar isso e lançar um erro tratado no `catch`, exibindo "Pokémon não encontrado" de forma amigável (na tela, não só no console).

Conceito: `Response` e `response.json()`

O que é?

`Response` é o objeto que `fetch` retorna assim que o servidor responde — ele contém informações como status (`response.status`), se deu certo (`response.ok`) e cabeçalhos, mas **ainda não** é o dado em si.

`response.json()` é o método que lê o corpo dessa resposta (que chega como texto no formato JSON) e o transforma em um objeto/array JavaScript.

Para que serve?

Entender essa etapa intermediária evita o erro comum de tentar usar os dados diretamente do `fetch`, sem "desembrulhar" com `.json()`.

Sintaxe:

```
const response = await fetch(url); // Response (ainda não são os dados)
console.log(response.status);      // 200, 404, etc
console.log(response.ok);          // true/false

const dados = await response.json(); // agora sim, os dados
```

Exemplo:

```
const response = await fetch("https://pokeapi.co/api/v2/pokemon/charizard");
console.log(response.status); // 200

const dados = await response.json();
console.log(dados.name); // "charizard"
```

■ **Armadilha comum:** `response.json()` também é assíncrono e retorna uma Promise — é fácil esquecer o `await` nessa segunda etapa e acabar tentando usar uma Promise como se fosse o dado final.

■ PARA FIXAR

JSON é texto estruturado; objeto JS é uma estrutura em memória.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Faça um `fetch` e, antes de chamar `.json()`, imprima `response.status` e `response.ok` no console.

■ Médio

Faça um `fetch` para uma URL propositalmente errada (ex: um pokémon que não existe, `"pokeapi.co/api/v2/pokemon/pokemon-inexistente-123"`) e imprima `response.status` (deve ser 404) e `response.ok` (deve ser `false`), sem chamar `.json()` ainda.

■ Avançado

Escreva uma função `buscarComTratamento(url)` que: faz o `fetch`, verifica `response.ok`, e só chama `response.json()` se a resposta foi bem-sucedida — caso contrário, lança um erro com uma mensagem baseada em `response.status`.

Conceito: JSON

O que é?

JSON (JavaScript Object Notation) é um **formato de texto** para representar dados estruturados — parecido com objetos e arrays JavaScript, mas é só texto (não código executável), usado como "linguagem universal" para APIs trocarem dados entre sistemas diferentes.

Para que serve?

É o formato mais comum de resposta de APIs. Entender a diferença entre "objeto JavaScript" e "texto JSON" evita confusão.

Sintaxe:

```
// JSON.stringify: converte objeto JS -> texto JSON (útil pra localStorage, Módulo 10)
const objeto = { nome: "Ana", idade: 22 };
const textoJSON = JSON.stringify(objeto);
console.log(textoJSON); // '{"nome":"Ana","idade":22}' (é uma STRING)

// JSON.parse: converte texto JSON -> objeto JS
const objetoDeVolta = JSON.parse(textoJSON);
console.log(objetoDeVolta.nome); // "Ana"
```

Diferença chave em relação ao objeto JS: no JSON, as chaves sempre vêm entre aspas duplas, e não pode conter funções, apenas dados (texto, número, boolean, null, array, objeto).

■ PARA FIXAR

JSON é texto estruturado; objeto JS é uma estrutura em memória.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie um objeto JS qualquer e converta para texto JSON com `JSON.stringify`. Imprima o resultado e confira, com `typeof`, que agora é uma `string`.

■ Médio

Pegue o texto JSON gerado no exercício anterior e converta de volta para objeto com `JSON.parse`. Acesse uma propriedade do objeto resultante para confirmar que funcionou.

■ Avançado

Explique, em um comentário, por que `response.json()` (do `fetch`) tem esse nome — o que ele está fazendo, em termos de `JSON.parse`, com o texto que o servidor enviou?

Conceito: Consumo de APIs (juntando tudo)

O que é?

A aplicação prática de tudo que foi visto: `fetch` → `response.json()` → dados → `map/filter` → DOM.

Exemplo completo — buscando e exibindo pokémons como cards:

```
<div id="catalogo"></div>
```

```
async function carregarPokemon(nome) {
  const response = await fetch(`https://pokeapi.co/api/v2/pokemon/${nome}`);
  const dados = await response.json();
  return dados;
}

async function renderizarCatalogo() {
  const nomes = ["pikachu", "charmander", "bulbasaur"];
  const catalogo = document.querySelector("#catalogo");

  const pokemons = await Promise.all(nomes.map((nome) => carregarPokemon(nome)));

  catalogo.innerHTML = pokemons
    .map((p) => `
    <div class="card">
      
      <h3>${p.name}</h3>
    </div>
  `)
    .join("");
}

renderizarCatalogo();
```

Note o uso de `Promise.all` — permite disparar várias requisições **em paralelo** em vez de uma por vez (você não precisa dominar isso agora, só reconhecer o padrão).

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Busque os dados de um pokémon via `fetch` e exiba o nome e a imagem (`dados.sprites.front_default`) dentro de uma `<div id="pokemon">` na página, usando `innerHTML`.

■ Médio

Crie um `<input id="buscaPokemon">` e um botão "Buscar". Ao clicar, busque o pokémon digitado via `fetch` e exiba seus dados na tela. Trate o caso de erro (pokémon não encontrado) mostrando uma mensagem amigável.

■ Avançado

Construa uma mini página de "catálogo de pokémons": busque 6 pokémons fixos (como no exemplo), renderize como cards com imagem e nome, e adicione um campo de busca que **filtra** (Módulo 4 — `filter` + `includes`) os cards já carregados pelo nome digitado, sem precisar de um novo `fetch` a cada tecla.

Conceito: API Key — o que é e quando você precisa de uma

O que é uma API Key?

Uma "chave" (geralmente uma string longa e única) que identifica quem está fazendo a requisição a uma API. Algumas APIs são totalmente abertas (como a PokéAPI, usada até aqui), outras exigem essa chave para controlar uso, cobrar por requisição, ou simplesmente saber quem está acessando.

Por que ela existe?

Para o provedor da API controlar quantas requisições cada pessoa/aplicação faz (evitar abuso), cobrar por uso em serviços pagos, e restringir acesso a dados sensíveis.

Onde ela normalmente fica?

Geralmente enviada como parte da URL (`?api_key=SUA_CHAVE`) ou em um cabeçalho da requisição (`headers: { Authorization: "Bearer SUA_CHAVE" }`).

Por que não devemos expor chaves secretas no frontend?

Todo código JavaScript que roda no navegador é **visível** para qualquer pessoa (basta abrir as ferramentas de desenvolvedor e olhar o código-fonte ou a aba de rede). Se você colocar uma API Key secreta diretamente no seu `script.js`, **qualquer visitante do site pode copiá-la** e usá-la em nome da sua conta — o que pode gerar cobranças indevidas ou uso indevido do seu limite de requisições.

Quando seria necessário um backend?

Quando a API exige uma chave secreta que não pode ser exposta, a solução é criar um pequeno servidor (backend) que guarda a chave em segredo, e o frontend faz requisições para **esse seu servidor**, que por sua vez faz a requisição para a API externa usando a chave, repassando só o resultado. Isso está fora do escopo desta trilha (que é 100% frontend), mas é importante saber que essa arquitetura existe — é o motivo pelo qual toda aplicação "de verdade" que usa chaves secretas normalmente tem um backend por trás.

Nem toda API Key é "secreta": algumas (como chaves de mapas, por exemplo) são feitas para serem públicas e têm outras formas de restrição (por domínio). Sempre leia a documentação da API para saber se a chave pode ficar no frontend com segurança ou não.

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Pesquise a documentação de uma API pública que exige chave (ex: OpenWeatherMap, usada no Projeto 3) e identifique: como se cadastra para obter a chave, e como ela deve ser enviada na requisição (URL ou header).

■ Médio

Cadastre-se gratuitamente em uma API que exija chave (ex: OpenWeatherMap) e faça uma requisição simples via `fetch`, passando a chave na URL, para confirmar que funciona.

■ Avançado

Escreva, em texto (não precisa ser código), um pequeno parecer explicando os riscos de publicar um projeto frontend no GitHub com uma API Key secreta diretamente no `script.js`. Que tipo de problema isso pode causar, e como algumas plataformas de hospedagem (como Vercel/Netlify) ajudam a mitigar isso com variáveis de ambiente e funções serverless (pesquisa livre, é só para gerar consciência do tema).

■ Desafio de revisão — Módulo 9

Construa uma busca de personagens usando a [Rick and Morty API](#) (pública, sem necessidade de chave):

1. Um `<input>` de busca e um botão.
2. Ao buscar, faça `fetch` em `https://rickandmortyapi.com/api/character/?name=NOME_DIGITADO`.
3. Trate o caso de "não encontrado" (a API retorna erro nesse caso) com `try/catch`.
4. Renderize os personagens encontrados como cards (imagem, nome, status) usando `map` + template literals.

Módulo 10 — Armazenamento no navegador

Pré-requisito: Módulos 1 a 9 (especialmente JSON e manipulação de arrays/objetos).

■ COMO ESTUDAR ESTE MÓDULO

Foco: localStorage é ótimo para pequenos dados persistentes no navegador. Lembre-se sempre: ele armazena strings, então arrays/objetos passam por JSON.stringify/JSON.parse.

Prática: Dica: construa uma lista de tarefas persistente para realmente fixar o conceito.

Material: YouTube PT-BR: CFB Cursos — WebStorage, localStorage e sessionStorage.

[Abrir material recomendado](#)

Conceito: localStorage

O que é?

Uma **Web API** do navegador que permite salvar dados no computador do próprio usuário, de forma que eles **persistam** mesmo depois de fechar a aba ou o navegador (diferente de uma variável JS comum, que se perde ao recarregar a página).

Para que serve?

Guardar preferências do usuário (tema escuro, idioma), carrinho de compras, lista de tarefas, ou qualquer dado que deve continuar existindo entre visitas — sem precisar de um servidor/backend.

Sintaxe:

```
localStorage.setItem("chave", "valor"); // salva (sempre como string!)
localStorage.getItem("chave");          // lê (retorna string ou null)
localStorage.removeItem("chave");       // remove uma chave específica
localStorage.clear();                   // remove tudo
```

■■ **Armadilha comum e fundamental:** localStorage só guarda strings. Para salvar um array ou objeto, você precisa converter com **JSON.stringify** antes de salvar, e com **JSON.parse** depois de ler (lembra do Módulo 9?):

```
const tarefas = [{ texto: "Estudar", concluida: false }];

localStorage.setItem("tarefas", JSON.stringify(tarefas)); // salva como texto JSON

const tarefasSalvas = JSON.parse(localStorage.getItem("tarefas")); // volta a ser array
```

Exemplo completo:

```
function salvarTarefas(tarefas) {
  localStorage.setItem("tarefas", JSON.stringify(tarefas));
}

function carregarTarefas() {
  const dados = localStorage.getItem("tarefas");
  return dados ? JSON.parse(dados) : []; // se não existe nada salvo, retorna array vazio
}

let tarefas = carregarTarefas(); // ao carregar a página, recupera o que já existia
```

■ **Outra armadilha comum:** `localStorage.getItem("chave")` retorna `null` se a chave nunca foi salva. Tentar `JSON.parse(null)` **não gera erro** (retorna `null`), mas é uma boa prática sempre verificar antes, como no exemplo acima.

■ PARA FIXAR

Lembre-se da regra central: `localStorage` guarda strings.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Salve seu nome no `localStorage` com `setItem`. Recarregue a página e use `getItem` para ler e exibir o valor salvo em um `<p>`.

■ Médio

Crie um contador (Módulo 6) cujo valor é salvo no `localStorage` a cada clique, e recuperado (em vez de começar do zero) toda vez que a página é recarregada.

■ Avançado

Pegue a lista de tarefas do desafio de revisão do Módulo 7 e adicione persistência completa: toda vez que uma tarefa for adicionada, marcada como concluída ou removida, salve o array inteiro atualizado no `localStorage` (como JSON). Ao carregar a página, recupere as tarefas salvas em vez de começar com uma lista vazia. Esse exercício é a base do Projeto 1.

■ Desafio de revisão — Módulo 10

Construa uma tela de "preferências do usuário" simples:

- Um `<select>` de tema (claro/escuro) que aplica a classe correspondente no `<body>` (Módulo 7) e salva a escolha no `localStorage`.
- Um `<input>` de nome que, ao ser preenchido, salva automaticamente (evento `input`) no `localStorage` e mostra "Olá, [nome]!" em algum lugar da página.
- Ao recarregar a página, o tema e o nome salvos devem ser restaurados automaticamente, sem o usuário precisar preencher de novo.

Módulo 11 — Organização do JavaScript em módulos

*Pré-requisito: Módulos 1 a 10. Este é o último módulo antes dos projetos finais — ele muda **como você organiza** o código, não o que ele faz.*

■ COMO ESTUDAR ESTE MÓDULO

Foco: Módulos não são um assunto para decorar: são uma forma de organizar responsabilidades. Pense em 'quem cuida dos dados?' e 'quem cuida da tela?'.

Prática: Dica: pegue um projeto antigo e divida-o em arquivos pequenos sem mudar o comportamento.

Material: Material de apoio: volte ao projeto e pratique import/export; use a documentação MDN quando quiser aprofundar.

[Abrir material recomendado](#)

Conceito: `import / export`

O que é?

Sistema nativo do JavaScript moderno (ES6+) para dividir o código em **vários arquivos**, cada um responsável por uma parte da aplicação, conectados entre si.

Para que serve?

Conforme um projeto cresce, um único `script.js` gigante fica difícil de manter. Módulos permitem separar, por exemplo, as funções de "manipulação de dados" das funções de "manipulação do DOM", cada uma em seu próprio arquivo.

Sintaxe:

Arquivo `tarefas.js`:

```
export function adicionarTarefa(lista, texto) {
  return [...lista, { texto, concluida: false }];
}

export const CHAVE_STORAGE = "tarefas";
```

Arquivo `script.js`:

```
import { adicionarTarefa, CHAVE_STORAGE } from "./tarefas.js";

let tarefas = [];
tarefas = adicionarTarefa(tarefas, "Estudar módulos");
console.log(tarefas);
```

Como conectar no HTML: ao usar `import/export`, o `<script>` que os importa precisa ter o atributo `type="module"`:

```
<script type="module" src="script.js"></script>
```

■ **Armadilha comum:** abrir um arquivo HTML que usa `type="module"` diretamente pelo sistema de arquivos (`file:///...`) costuma dar erro de CORS no navegador. Módulos exigem que a página seja servida por um **servidor local** (ex: extensão "Live Server" do VS Code, ou `npx serve`).

Duas formas de exportar:

```
// exportação nomeada (várias por arquivo, precisa do nome exato ao importar)
export function minhaFuncao() {}

// exportação padrão (uma por arquivo, pode importar com qualquer nome)
export default function minhaFuncaoPrincipal() {}
// import minhaFuncaoPrincipal from "./arquivo.js";
```

■ PARA FIXAR

Tente explicar este conceito com suas próprias palavras e crie um exemplo mínimo antes de continuar.

Teste rápido: consigo explicar este conceito sem olhar a definição?

■ Minhas anotações

Exercícios

■ Simples

Crie dois arquivos: `matematica.js` (com uma função `somar(a, b)` exportada) e `script.js` (que importa e usa essa função). Configure o `<script type="module">` no HTML e confirme que funciona (lembre-se de usar um servidor local).

■ Médio

Separe um script existente (ex: o desafio do Módulo 6 — calculadora de IMC) em dois arquivos: um `imc.js` com a função pura de cálculo (`calcularIMC(peso, altura)`, exportada) e um `script.js` com a lógica de DOM/eventos, que importa essa função.

■ Avançado

Pegue o projeto de lista de tarefas (Módulos 7 e 10) e reorganize em pelo menos 3 arquivos: `storage.js` (funções de salvar/carregar do `localStorage`), `tarefas.js` (funções puras de manipulação do array de tarefas: adicionar, remover, marcar concluída) e `script.js` (DOM, eventos, e orquestração — importando dos outros dois). Essa separação entre "lógica de dados" e "lógica de tela" é uma prática real usada em projetos profissionais.

■ Desafio de revisão — Módulo 11

Reorganize qualquer um dos desafios anteriores (sua escolha) separando claramente:

- Um arquivo só com funções que **não tocam no DOM** (recebem dados, retornam dados).
- Um arquivo principal que cuida de `querySelector`, eventos e renderização, importando as funções do primeiro arquivo.

Esse padrão de separação é exatamente o que você vai usar nos projetos finais a seguir.

■ ■ Projetos para consolidar o aprendizado

Estes projetos não têm solução pronta nesta apostila — a ideia é você aplicar tudo que aprendeu, revisitando os módulos sempre que precisar. Cada projeto lista os requisitos funcionais e os conceitos que ele deve exercitar, não o código pronto.

Projeto 1 — Lista de tarefas (To-Do List)

Conceitos envolvidos: DOM, eventos, arrays, objetos, funções, `localStorage`.

Requisitos:

- [] Formulário para adicionar uma nova tarefa (texto).
- [] Cada tarefa é um objeto `{ texto, concluida }`, guardado em um array.
- [] Checkbox (ou clique na tarefa) para marcar como concluída, com estilo visual diferenciado (ex: texto riscado, via `classList`).
- [] Botão para remover uma tarefa individual.
- [] Contador mostrando quantas tarefas estão pendentes.
- [] Persistência com `localStorage`: ao recarregar a página, as tarefas continuam lá.
- [] Validação: não permitir adicionar tarefa com texto vazio.

Vai além (opcional): filtro para mostrar "todas / pendentes / concluídas".

Projeto 2 — Catálogo com filtros

Conceitos envolvidos: arrays, objetos, `map`, `filter`, `find`, DOM, eventos, busca, filtros.

Requisitos:

- [] Um array fixo (no próprio código, sem API ainda) de pelo menos 10 produtos, cada um com `nome`, `preco`, `categoria` e `disponivel`.
- [] Renderização de todos os produtos como cards na tela (`map` + template literals ou `createElement`).
- [] Campo de busca por nome (evento `input` + `filter` + `includes`, Módulo 6).

- [] `<select>` de categoria que filtra os produtos exibidos (evento `change` + `filter`).
 - [] Checkbox "mostrar apenas disponíveis" que também filtra a listagem.
 - [] Todos os filtros devem poder ser combinados ao mesmo tempo (busca + categoria + disponibilidade).
 - [] Mensagem amigável quando nenhum produto corresponde aos filtros.
-

Projeto 3 — Dashboard de clima

Conceitos envolvidos: HTML, CSS, JavaScript, `fetch`, `async/await`, JSON, API externa, DOM, tratamento de erros.

Sugestão de API: [OpenWeatherMap](#) (exige cadastro gratuito e API Key — releia a seção sobre API Keys do Módulo 9 antes de começar).

Requisitos:

- [] Campo para o usuário digitar o nome de uma cidade.
 - [] Ao buscar, fazer `fetch` para a API de clima com `async/await`.
 - [] Exibir: temperatura atual, sensação térmica, descrição do clima (ex: "céu limpo") e um ícone/emoji correspondente.
 - [] Tratamento de erro com `try/catch`: cidade não encontrada deve mostrar uma mensagem amigável, não travar a página nem mostrar erro no console apenas.
 - [] Indicador de "carregando..." enquanto a requisição está em andamento.
 - [] (Opcional) Salvar a última cidade buscada no `localStorage` e carregá-la automaticamente ao abrir a página.
-

Projeto 4 — Catálogo de personagens

Conceitos envolvidos: API externa, `fetch`, `async/await`, cards dinâmicos, pesquisa, filtros, DOM, eventos.

Sugestão de API: [Rick and Morty API](#) ou [PokéAPI](#) — ambas públicas e sem necessidade de chave.

Requisitos:

- [] Ao carregar a página, buscar e exibir uma lista inicial de personagens (cards com imagem, nome e outros dados relevantes).
 - [] Campo de busca que faz uma nova requisição à API filtrando pelo nome digitado (a maioria dessas APIs aceita um parâmetro de busca na própria URL).
 - [] Tratamento de erro para buscas sem resultado.
 - [] Clique em um card deve mostrar mais detalhes daquele personagem específico (pode ser em um modal, reaproveitando o Módulo 7, ou uma nova área da página).
 - [] Estado de carregamento visível enquanto os dados não chegam.
-

Projeto 5 — Projeto livre

Conceitos envolvidos: praticamente todos os anteriores, à sua escolha de combinação.

Requisitos mínimos (a estrutura é livre, mas deve conter):

- [] Pelo menos uma fonte de dados dinâmica: um array próprio manipulado com `map/filter/find`, ou uma API externa consumida via `fetch`.
- [] Interface totalmente gerada/atualizada via DOM (sem HTML estático fingindo ser dinâmico).
- [] Pelo menos um formulário com validação.
- [] Persistência de algum dado relevante com `localStorage`.
- [] Código organizado em pelo menos 2 arquivos JS usando `import/export`.
- [] Tratamento de erros nas operações assíncronas, se houver.

Ideias de tema (escolha uma ou crie a sua): lista de filmes favoritos, calculadora de gastos mensais, gerador de senhas com validações, agenda de contatos, enquete/votação com resultados em tempo real, cardápio de restaurante com carrinho, biblioteca pessoal de livros.

■ Checklist de aprendizado

```
[ ] Variáveis (let, const)
[ ] Tipos de dados
[ ] Operadores
[ ] Condicionais (if/else)
[ ] Loops (for, while, for...of)
[ ] Funções (declaration e arrow)
[ ] Arrays
[ ] Objetos
[ ] Destructuring
[ ] Spread operator
[ ] Template literals
[ ] Métodos de strings
[ ] map
[ ] filter
[ ] find
[ ] forEach
[ ] includes
[ ] DOM (o que é e como se diferencia da linguagem JS)
[ ] querySelector / querySelectorAll
[ ] getElementById
[ ] textContent / innerHTML
[ ] value
[ ] createElement / appendChild
[ ] classList
[ ] Eventos (addEventListener)
[ ] click / input / change / submit
[ ] event / event.preventDefault()
[ ] Formulários
[ ] Manipulação de classes CSS via JS
[ ] Criação dinâmica de elementos
[ ] Funções assíncronas
[ ] Promise
[ ] async / await
[ ] try/catch
[ ] fetch / Response / response.json()
[ ] JSON
[ ] Consumo de APIs
[ ] API Keys (o que são e por que cuidar delas no frontend)
[ ] localStorage
[ ] import / export
[ ] Organização de código em módulos
[ ] Projeto 1 – Lista de tarefas
[ ] Projeto 2 – Catálogo com filtros
[ ] Projeto 3 – Dashboard de clima
[ ] Projeto 4 – Catálogo de personagens
[ ] Projeto 5 – Projeto livre
```

Fim da trilha. Bons estudos — e lembre-se: releia os módulos anteriores sempre que um conceito novo depender deles, e priorize sempre tentar resolver os exercícios antes de buscar qualquer solução pronta.